

Assessing and Improving Prompting Large Language Models for Software Vulnerability Analysis

YU NONG, University at Buffalo, SUNY, USA

GUANGBEI YI, Washington State University, USA

MOHAMMED ALDEEN, Clemson University, USA

LONG CHENG, Clemson University, USA

HONGXIN HU, University at Buffalo, SUNY, USA

HAIPENG CAI*, University at Buffalo, SUNY, USA

Large language models (LLMs) have demonstrated potential in diverse domains including software analysis. Yet there is a lack of systematic *assessment* of how LLMs perform in comparison to various extant approaches, and how LLMs may be *improved*, for software vulnerability analysis, via prompt engineering. In this paper, we present a comprehensive, large-scale *empirical study* of ten LLMs with seven prompting strategies versus nine traditional (five code-analysis- and four deep-learning-based) techniques on three vulnerability analysis tasks (detection, classification, and repair) against five real-world datasets (8,000+ C/C++ samples, including a zero-day dataset). We show that, with existing prompting strategies, LLMs often struggle with practical vulnerability analysis and underperform the traditional approaches. Via in-depth case analysis, we reveal that the evaluated LLMs frequently suffer from incorrect reasoning. Based on these findings, we improve the prompting with a vulnerability-specific adaptation of chain-of-thought (CoT), named Vulnerability-Semantics-guided Prompting (VSP). Our results show that VSP improves the performance of some of the LLMs in certain configurations across the three tasks. VSP also helps mitigate the reasoning limitations for some of the evaluated LLMs. For vulnerability detection on unseen data, improvements are limited or marginal for some models. We further identify seven common challenges that led to the LLMs' incorrect answers in these tasks and provide actionable recommendations to help mitigate them.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Theory of computation** → **Program analysis**.

Additional Key Words and Phrases: Vulnerability analysis, large language models, prompting, vulnerability semantics, vulnerability detection, vulnerability classification, vulnerability repair

1 Introduction

Software vulnerabilities are increasingly prevalent [41, 49, 52, 76] and consequential [23, 27, 40]. In response, numerous techniques for a range of defensive tasks (i.e., *detecting* the presence of vulnerabilities [13, 13, 22, 34, 48, 51, 65, 90, 91], *classifying* vulnerabilities of various types [12, 30, 37, 45, 50, 60, 64, 81], and *repairing* the vulnerabilities found [17, 29, 32, 58, 75, 102]) have been proposed. Tackling these *three essential software*

*Haipeng Cai is the corresponding author.

Authors' Contact Information: Yu Nong, yunong@buffalo.edu, University at Buffalo, SUNY, Buffalo, USA; Guangbei Yi, guangbei.yi@wsu.edu, Washington State University, Pullman, USA; Mohammed Aldeen, mshujaa@g.clemson.edu, Clemson University, Clemson, USA; Long Cheng, lcheng2@clemson.edu, Clemson University, Clemson, USA; Hongxin Hu, hongxinh@buffalo.edu, University at Buffalo, SUNY, Buffalo, USA; Haipeng Cai, haipengc@buffalo.edu, University at Buffalo, SUNY, Buffalo, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/6-ART

<https://doi.org/10.1145/3821416>

vulnerability analysis tasks, existing approaches, including traditional ones based on code analysis [12, 22, 37, 45, 48, 64, 81] and more recent ones based on deep learning (DL) [17, 28, 29, 38, 53, 54, 60, 82, 95, 105], have been both successful and subject to various limitations [9, 15, 42, 67, 69, 71].

Lately, pre-trained large language models (LLMs) have demonstrated potential in diverse domains including software analysis [25, 43, 44, 57, 97, 103]. Some LLM-based techniques have also been proposed for vulnerability analysis [36, 46, 80, 83, 96]. However, these techniques require assistance from external information (e.g., error messages), which may not always be available [46, 80, 96]. Other works fine-tune the open-source LLMs [36, 83], which may require considerable resources and cannot use the most powerful commercial LLMs. These constraints limit the application of LLMs for vulnerability analysis. Therefore, a direct and easy-to-use LLM-based approach for vulnerability analysis is needed. Among various LLM-based strategies, prompt engineering stands out as a highly accessible and practical direction, especially given that pre/re-training or fine-tuning may not be feasible for most users, particularly with commercial LLMs. Yet there is still a lack of comprehensive assessment on how LLMs perform across different vulnerability analysis tasks with different prompting strategies and how to improve them.

Existing studies [21, 75, 84, 87, 104] have taken important steps in this regard. However, they are subject to one or more of the following limitations: (1) address single tasks only (almost all focusing on vulnerability detection), (2) use relatively small datasets (e.g., <400 samples in total [84, 87] or 30 real-world samples [87]), (3) dismiss traditional (i.e., non-LLM-based) approaches, including code analysis- and DL-based ones, in comparisons [75, 84, 104], (4) provide no evaluation on LLM reasoning correctness, and (5) ignore the data leakage/contamination issues [75, 84, 104] (i.e., the LLMs may have seen the testing samples' labels during their training). Moreover, while these studies generally conclude that current LLMs are not yet ready for vulnerability analysis, they (6) only directly use existing prompting strategies without providing or evaluating improvement strategies.

In this paper, we seek to fill these gaps through a comprehensive and large-scale assessment of LLMs with different prompting strategies, and exploration of how to improve them, for vulnerability analysis across three tasks (detection, classification, and repair) against more than 8,000 real-world code samples from four existing datasets and a zero-day dataset (where all the vulnerabilities were released after all the LLMs' training cut-off dates).

In this study, we seek answers to seven research questions:

- RQ1:** (How well) can current LLMs detect, classify, and repair vulnerabilities with existing prompting strategies?
- RQ2:** (How much) can we improve the prompting strategies for LLMs in vulnerability analysis?
- RQ3:** How do LLM configurations and system prompts affect the effectiveness of vulnerability analysis tasks?
- RQ4:** How do LLMs compare to traditional (DL- and code-analysis-based) approaches to the same three essential vulnerability analysis tasks?
- RQ5:** Are the vulnerability analysis results produced by LLMs based on correct reasoning about the vulnerabilities?
- RQ6:** Can we improve the reasoning of LLMs in vulnerability analysis via prompt engineering?
- RQ7:** What are the common challenges to LLMs in analyzing real-world software vulnerabilities?

To answer these questions, we assess ten LLMs with five prompting strategies (standard prompting, standard few-shot learning, retrieval-augmented generation (RAG), naive chain-of-thoughts (CoT) [92], and zero-shot completion [75]) against nine SOTA traditional (five code-analysis-based and four DL-based) approaches. Then, we examine whether the generated reasoning from LLMs captures the *vulnerability semantics* (i.e., behaviors of a program that make it vulnerable) and justifies their responses correctly. Based on the limitations found, we propose a vulnerability-specific CoT adaptation, named *Vulnerability-Semantics-guided Prompting* (VSP). It features task-specific prompting unified across the three tasks via vulnerability semantics, which is represented by the vulnerability-relevant code semantics of a program [7] that capture (i.e., account for) its vulnerable behaviors.

Accordingly, VSP uses descriptions of vulnerability-relevant data and control dependencies as exemplars to encourage the model to focus its reasoning on the vulnerability behavior when producing an answer. We also assess the challenges faced by LLMs for vulnerability analysis which lead to incorrect predictions. We further provide actionable recommendations for mitigating these challenges such as leveraging traditional code analysis techniques for control/data flow analysis.

Among other findings, our study reveals that:

- Direct use of existing prompting strategies on LLMs does not work well for vulnerability analysis (with up to 47.06–54.55% F1 for detection, 25.40–41.11% micro F1 for classification, and 20.40–70.83% correct rate for repair) and sometimes *underperform* the traditional approaches (which achieve up to 52.31% F1, 36.23% micro F1, and 83.33% correct rate for detection, classification, and repair, respectively). Against the zero-day dataset, some LLMs perform much worse than they perform on existing datasets, indicating their potential subjection to data leakage/contamination issues. This underscores the critical need for better prompting strategies specific to vulnerability analysis tasks and for using zero-day datasets as the primary benchmark for unbiased evaluation to mitigate data leakage/contamination risks to evaluation validity.
- The LLMs suffer from incorrect reasoning, where only 4.03–14.07% of the reasoning is correct; even *among the correct (final) predictions*, 16.51–69.26% of the reasoning is incorrect, which limits LLMs' usability and credibility for vulnerability analysis. Meanwhile, (i) the reasoning underlying incorrect responses is consistently incorrect, while (ii) for producing correct responses, different tasks depend on reasoning correctness to different extents—more challenging tasks (e.g., vulnerability repair) have stronger reliance on correct reasoning for producing correct predictions than less challenging tasks (e.g., vulnerability detection). In other words, more challenging tasks are less likely to suffer from *coincidental correctness* of their LLM responses, while LLMs can be more likely to produce coincidentally correct answers for less challenging vulnerability analysis tasks.
- With our proposed vulnerability-specific adaptation of Chain-of-Thought (CoT) prompting (i.e., VSP), some of the LLMs show improved performance for the three tasks in specific evaluated scenarios (achieving F1 scores of 48.70–62.68% for detection, micro F1 of 40.16–47.27% for classification, and correct rates of 30.25–83.33% for repair). VSP also enables certain top-performing LLMs to outperform the evaluated traditional baselines (by up to 25.80% for detection, 234.11% for classification, and 6.10% for repair) and helps mitigate incorrect reasoning (achieving 50.15–94.22% correct reasoning rates for correct final predictions). Our findings show that VSP is associated with improved performance in certain settings, consistent with its design goal of emphasizing vulnerability-focused semantic reasoning.
- Seven common causes can lead to LLMs' *incorrect predictions* across the three tasks, including *insufficient context* (e.g., LLMs do not know about a self-defined/external function called) and *incomplete data/control flow analysis* (e.g., LLMs miss key data-flow facts as part of the vulnerability semantics) as primary causes; in other cases, LLMs suffer from not following exemplars, logic errors, and unreadable output issues. These novel insights into why LLMs can fail in vulnerability analysis tasks provide informative guidance for developing future LLM-based vulnerability analysis techniques.

Based on these findings, we provide actionable suggestions for improving LLM-based vulnerability analysis. Among others, we recommend (1) using vulnerability-semantics-guided prompting strategies, (2) using LLMs that are capable of following vulnerability-analysis-specific task instructions, especially the exemplars for effective in-context learning, as well as understanding code semantics for vulnerability reasoning, (3) providing necessary context information *on demand* during prompting to both meet the contextualization needs for vulnerability analysis and reduce the total prompt length, and (4) integrating traditional code analysis techniques for advanced LLM-based vulnerability analysis to mitigate LLMs' insufficiency in data/control flow analysis essential for vulnerability-related code semantics reasoning.

Table 1. LLMs examined in our study

Model	Version	#Params	Max #Tokens	Vendor	Release date	Cutoff Date	License
Gemini	Gemini-Pro	50T	30,720	Google	Dec. 2023	Apr. 2023	Commercial
Claude3	Claude-3-Sonnet	70B	200,000	Anthropic	Mar. 2024	Aug. 2023	Commercial
GPT-3.5	gpt-3.5-turbo-16k	175B	16,385	OpenAI	June 2023	Jan. 2022	Commercial
GPT-4	gpt-4o	175B+	128,000	OpenAI	May 2024	Oct. 2023	Commercial
Llama2-7b	llama-2-7b-chat-hf	7B	4,096	Meta	July 2023	Sep. 2022	Open-Source
Llama2-13b	llama-2-13b-chat-hf	13B	4,096	Meta	July 2023	Sep. 2022	Open-Source
Falcon	falcon-7b-instruct	7B	2,048	TII	June 2023	Apr. 2023	Open-Source
CodeLlama	codellama-13b-instruct-hf	13B	4,096	Meta	July 2023	Sep. 2022	Open-Source
Qwen2.5-Coder	qwen2.5-coder-14b-instruct	14B	131,072	Alibaba	Sep. 2024	Oct. 2023	Open-Source
DeepSeek-Coder-V2	deepseek-coder-v2-lite-instruct	16B	131,072	DeepSeek	July 2024	Oct. 2023	Open-Source

To the best of our knowledge, our work is the first to systematically assess the prompting strategies of LLMs across three essential vulnerability analysis tasks at large scale and compare them with various SOTA traditional approaches to the same tasks, while exploring and evaluating an improving methodology. While LLMs face major challenges for analyzing (especially zero-day) vulnerabilities (i.e., undesirable effectiveness in both prediction accuracy and reasoning correctness), our findings highlight/point to a direction for improving the LLMs for that purpose. The substantial gaps in LLMs' performance on existing vulnerability (i.e., with labels likely leaked) versus zero-day (i.e., vulnerability labels unseen by models) datasets point to the critical need for future LLM-based vulnerability analysis to be evaluated against testing samples labeled after the model's cutoff date.

Our datasets and experimental results are available at <https://doi.org/10.5281/zenodo.20585652>.

2 Methodology

In this section, we describe our study setup, including the *LLMs* (§2.1) examined, the vulnerability analysis *tasks* (§2.2) targeted (i.e., goal/motivation and formulation of each task), the *datasets* (§2.3) utilized, the *prompting strategies* (§2.4) considered (existing ones and the improved one), and the *baseline traditional approaches* to vulnerability analysis (§2.6) as we compare LLMs with.

2.1 LLMs Assessed

Table 1 summarizes the ten LLMs chosen for our study to cover the models with a variety of sizes, vendors, and specializations. **Gemini** [85] and **Claude3** [4] are the LLMs from Google and Anthropic which can understand and generate natural language and code. We chose their most popular versions (Pro and Sonnet) for both effectiveness and efficiency. **GPT-3.5** [26] is a primary LLM from OpenAI for general purposes, and we chose the Turbo-16k version to allow more input/output tokens. We also consider **GPT-4** which is the more powerful LLM from OpenAI and use its most widely-used version GPT-4o. **Llama2** [86] is an open-source LLM from Meta, of which we chose to deploy 7b and 13b versions locally due to our resource limitations. For the same reason, we chose the 7b-instruct version of **Falcon** [107], an open-source LLM from the Technology Innovation Institute (TII).

To provide a comprehensive assessment of LLMs' capabilities in software vulnerability analysis, we also include three code-specific LLMs. **CodeLlama** [79], developed by Meta, is designed to generate and discuss code through text prompts, making it a valuable tool for various programming needs. **Qwen2.5-Coder** [39], from Alibaba, is a code-specific model for code generation, reasoning, and fixing. **DeepSeek-Coder-V2** [106] is an open-source model from DeepSeek that offers powerful performance in code-specific tasks.

Finally, to investigate the potential of LLMs beyond prompt engineering, we also evaluate the capability of fine-tuned LLMs. We specifically select Llama2-13b for fine-tuning, as it demonstrated the strongest overall performance among the evaluated open-source models in our preliminary experiments. By fine-tuning this top-performing open-source baseline on our datasets, we aim to rigorously compare the effectiveness of our proposed prompting strategy against the fine-tuning approach.

2.2 Tasks

Our study addresses three vulnerability analysis tasks: (1) **detection** (determining if a given code sample is vulnerable or not), (2) **classification** (classifying a vulnerable code sample into a vulnerability type/class (CWE)), and (3) **repair** (fixing a vulnerable sample based on the vulnerability type and location). These tasks span various levels of analysis, representing the key steps in holistic defenses against software vulnerabilities. Note that we aim to assess LLMs for these tasks separately—each assumes what it needs independently, not taking the output of a previous task as its input.

2.2.1 Task 1: Vulnerability Detection. In this task, we aim to check the presence of vulnerabilities in a given code sample, a typical defensive vulnerability analysis [15, 28, 105]. This task serves as a starting point for evaluating whether LLMs can recognize vulnerabilities in a broader sense. Note that in this task, the goal is not to detect a specific vulnerability but to determine whether the code contains a vulnerability of any kind. This mirrors real-world vulnerability scanning, where tools are expected to identify any security flaw in a piece of code, regardless of the category of that flaw.

We *formulate* this as a binary classification task. In the input (i.e., prompt) to LLMs, we start with the question “Q: Does the following code have exploitable vulnerabilities?”. We also request the model: “Please check whether the vulnerability, if any, can be practically exploited to cause real security consequences.” This guides the model to detect *exploitable* vulnerabilities rather than just potential/general bugs. The prompt ends with the code under analysis.

2.2.2 Task 2: Vulnerability Classification. The *goal* of this task is to, after determining a code sample as vulnerable, further identify the type of vulnerability (in the form of a CWE ID) in it. In practical vulnerability analysis, developers not only need to know whether given code is vulnerable but also the vulnerability type to inform their responses and countermeasure decisions [60].

We *formulate* this as a multi-class classification task. The input to LLMs starts with the question: “Q: The following code has been confirmed as vulnerable, which CWE does it have?”, followed by the code sample under analysis. We also provide the CWE options for the models to choose. That is, the models only need to classify a vulnerability into one of the five CWEs. The reason is that the CWE options are at different levels of abstraction and are related to one another (e.g., CWE-20 is a ‘super type’ of several specific input validation issues, such as SQL Injection CWE-89, XSS CWE-79, etc). By providing the options, we can avoid the conflicts on the CWE abstraction levels. Another reason is that state-of-the-art techniques (e.g., VulChecker [60]) also classify vulnerabilities into one of the given CWE options. To compare LLMs with these (non-LLM-based) techniques fairly, we follow the same approach to this *scoped* vulnerability classification task. This scoped task formulation also facilitates automatically evaluating LLMs’ response correctness: given the scope of CWE choices, the LLMs never classified a vulnerable code sample into CWEs that are at higher/lower levels than the ground-truth labels.

2.2.3 Task 3: Vulnerability Repair. The *goal* of this task is to eliminate the vulnerability from given vulnerable code while preserving its functionalities. It is a crucial vulnerability-analysis step as it addresses security risks induced by the vulnerabilities hence protecting the respective software from potential threats/attacks [29, 75].

We *formulate* repair as a text-to-text transformation task. The LLM prompt starts with the question: “Q: The following code has a <CWE ID> vulnerability at line <line no.>. Please provide a valid patch.” We also request the model: “only show the code changes needed rather than the entire patched code, to conform to the conventional patch format.” The prompt then ends with the vulnerable code. We explicitly provide the vulnerability location (via the line no.) and type (via the CWE ID), which is common in bug/vulnerability repair [17, 29, 32].

2.3 Datasets

We evaluate the chosen LLMs on the three tasks using real-world C/C++ code as *testing* data. We focus on C/C++ language because (1) C/C++ is the most vulnerable language, and (2) many critical software systems are written in C/C++ (e.g., Linux kernel, Qemu, and FFmpeg). Our choice of C/C++ is also aligned with the availability of existing vulnerability datasets (with quality ground truth labels); such datasets for other languages are relatively rare compared to C/C++. We selected datasets containing low-level memory vulnerabilities (e.g., buffer overflows, use-after-free) which represent a class of vulnerabilities that are both highly prevalent and notoriously difficult to address with existing techniques. These types of vulnerabilities, which often involve intricate memory management and pointer arithmetic, are a significant challenge for both traditional tools and modern approaches, including LLMs. Specifically, within each dataset, we focused on five prevalent memory-related CWEs, with slight variations in the exact CWEs selected. The categories of these vulnerabilities that we target are mostly in the list of top-25 most dangerous CWEs [1].

VulChecker Dataset. For Tasks 1 and 2, we use the dataset from VulChecker [60] because the samples are compilable (hence allowing us to directly compare LLMs with VulChecker) and fairly diverse (covering 19 real-world projects, 35 CVEs, and 5 CWEs—CWE-121, 122, 190, 415, and 416). In this dataset, each vulnerable project version has one or more vulnerable points each labeled with one of the 5 CWEs. Given its source size, a project version cannot be fed as a whole to LLMs at once. Thus, we consider individual functions as input units as commonly adopted in DL-based vulnerability analysis [15, 28, 54, 55, 105]. To that end, we extract 85 vulnerable functions (11 CWE-121, 15 CWE-122, 40 CWE-190, 7 CWE-415, and 12 CWE-416). For Task 1, we also extract 87,353 non-vulnerable functions from these projects. Since there are too many non-vulnerable functions that cause extreme data imbalance (>1000:1), we follow VulChecker to downsample the non-vulnerable functions to 1%, resulting in 873 ones [60]. Nevertheless, we still kept the dataset imbalanced because non-vulnerable samples are still more prevalent than vulnerable samples in the real-world vulnerability distribution [15, 60], and such imbalanced dataset setting is widely used in previous works [15, 28, 54, 55].

ExtractFix Dataset. For Task 3, we use the dataset from ExtractFix [32] because the code therein is compilable and comes with test cases that allow us to validate patches. From this dataset, we collected 24 vulnerabilities that can be reproduced, for each of which we label the vulnerable line and the CWE ID based on the crash information from the test cases. These 24 also cover 5 CWEs (7 CWE-787, 9 CWE-125, 2 CWE-190, 4 CWE-369, and 2 CWE-476). Again, we use function-level samples.

BigVul+CVEFixes. However, the two datasets are relatively small for a large-scale study. Thus, we further build a larger dataset where the samples are collected from BigVul [24] and CVEFixes [11], two of the most widely used real-world vulnerability datasets [28, 29, 38, 68, 70]. We select the vulnerable samples with the same CWEs as in VulChecker and ExtractFix, resulting in 938 vulnerable samples (149 CWE-121, 159 CWE-122, 333 CWE-190, 112 CWE-415, and 185 CWE-416) for Tasks 1 and 2, as well as 236 vulnerability repair samples (126 CWE-787, 48 CWE-125, 43 CWE-190, 6 CWE-369, and 13 CWE-476) for Task 3. For Task 1, we also select the non-vulnerable samples from BigVul and downsample to 5,892 samples to avoid the extreme imbalance. We remove any overlap between BigVul and CVEFixes to avoid duplicates.

Zero-Day Dataset. However, the datasets above include samples from many years ago. It is possible that these samples were used to train the LLMs, causing potential data leakage/contamination issues. To mitigate this, we constructed a Zero-Day dataset where all the vulnerabilities were reported after the latest cutoff date of the LLMs evaluated (October 2023 from GPT-4o). The construction process was as follows:

- (1) **CVE Collection:** We first developed a crawler to automatically collect the latest CVEs from the NVD website [63].

- (2) **Date and CWE Filtering:** We then filtered these collected CVEs to include only those with a publishing date after the LLMs' latest training cutoff date (October 2023) and that correspond to the most common five CWEs used in BigVul+CVEFixes (CWE-787, CWE-125, CWE-190, CWE-416, and CWE-476).
- (3) **Commit Link Verification:** We further filtered for CVE reports that provided a direct commit link for patching the vulnerability. This step was essential to ensure the availability of both the vulnerable source code and the ground truth patches, which are necessary for our evaluation.
- (4) **Function-Level Sample Extraction:** Finally, from the collected commits, we manually extracted function-level samples written in C/C++. For each relevant commit, we checked the file types (e.g., .c, .cpp) of the changed code. We then manually filtered out changes irrelevant to the vulnerabilities (e.g., documentation updates, release notes) and extracted the vulnerable functions both before and after patching.

With the process, we collected a Zero-Day dataset with 103 vulnerabilities which also cover five CWEs (18 CWE-787, 26 CWE-125, 11 CWE-190, 24 CWE-416, 24 CWE-476) for the three tasks. The vulnerable and fixed versions of these samples are used for Task 1, while the vulnerable versions are used for Tasks 2 and 3.

PrimeVul Dataset. Recently, Ding et al. [21] point out the issue of low label accuracy in the previous datasets (e.g., BigVul and CVEFixes), where up to 75% of the sample labels are inaccurate. To address this, they introduced PrimeVul, improving label accuracy to 92%. Given this higher quality, we employ PrimeVul to evaluate LLMs on vulnerability analysis tasks. We again select the five most common CWEs in BigVul+CVEFixes (CWE-121, CWE-122, CWE-190, CWE-416, and CWE-476), and select 10% of the non-vulnerable samples from its testing set (considering the cost of LLMs and the balance of the dataset). As a result, the testing set we use from PrimeVul comprises a total of 2,635 function-level samples, of which 72 are vulnerable, which we call PrimeVul-Subset. The 72 vulnerable samples cover 1 CWE-121, 7 CWE-122, 14 CWE-190, 12 CWE-415, and 38 CWE-416. For Task 1, all 2,635 samples from the PrimeVul testing set are utilized. For Task 2 and Task 3, only the 72 vulnerable samples are used.

Besides, to address the limited representativeness of a restricted CWE subset and to facilitate broader cross-referencing, we also evaluate on the PrimeVul dataset with all the CWEs included in its testing dataset. This results in 435 vulnerable samples spanning 62 CWEs. To balance the dataset, we include 15,920 non-vulnerable samples. For Task 1, the respective vulnerable and non-vulnerable sets are utilized. For Tasks 2 and 3, only the vulnerable samples are used. We refer to this dataset as PrimeVul-Full in the evaluation. For the distribution of the CWEs, refer to the original PrimeVul paper [21].

```
// Line-by-line code summarization (used in Naive CoT)
The cycle0 function first initializes two integer variables 'v' and
'w' and modifies 'v' via a function call to 'set'. Next, it
retrieves the number of offers from the hello object. Then, it
declares an array offered of size 'max_offered + 1' and iterates
over the number of offers, fetching each offer and storing it in
the offered array while updating 'w' by adding 'v' in each
iteration. After that, it sends the final value of 'w' using the
'send' function and calls the 'reset' function to reset some state.
```

Fig. 1. Reasoning steps (describing the *functionality semantics*) in naive CoT (for the example in Fig. 2).

each task, resulting in training sets of 330,492 samples for vulnerability detection, 16,109 for classification, and 12,808 for repair.

2.4 Prompting strategies

In this study, we study different prompting strategies to evaluate the inherent capabilities of LLMs and their responsiveness to semantic guidance.

DiverseVul Dataset. To evaluate the capabilities of fine-tuned LLMs, we utilize the DiverseVul [16] dataset, a large-scale collection of real-world vulnerable code. To prevent data leakage and ensure the validity of our evaluation, we filtered the dataset to remove any samples that overlap with our testing datasets (VulChecker, ExtractFix, BigVul+CVEFixes, Zero-Day, and PrimeVul). To ensure the quality of the fine-tuning, we utilized all available samples for

2.4.1 Existing Prompting strategies. We examine five popular/well-known strategies for prompting LLMs.

Standard Prompting. We ask the LLMs to solve a task without providing any exemplars or guidance, aiming to assess their raw/original capabilities for the task.

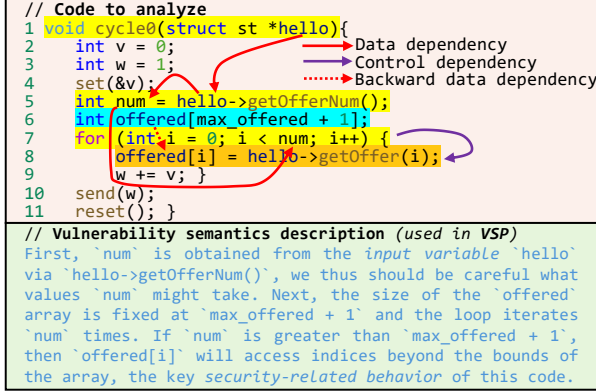


Fig. 2. Illustration of *vulnerability semantics* (as opposed to *functionality semantics* shown in Fig. 1) on a buffer-overflow code sample, with the approximation (slice) highlighted at the top and description at the bottom. The sensitive statement is marked in orange; the relevant contextual statements which make the sensitive statement reachable from the external inputs are marked in yellow, and those it data-depends on in cyan. The approximating data/control dependencies are denoted with →.

ing issues like insufficient context and improving the quality of few-shot learning [33]. For our RAG experiments, given a testing sample, we first use the BM-25 algorithm [77] to find the most similar samples from a predefined exemplar set, as the BM-25 algorithm is very fast with good performance for code comparison [20, 31, 62]. Similar to standard few-shot learning, we select a total of 10 exemplars for each prompt. These retrieved samples are then used as exemplars for standard few-shot learning, meaning that we provide the questions, code samples, and final answers of the exemplars in the prompt. To ensure diversity and avoid data leakage within the same dataset during retrieval, we cross-use our datasets (VulChecker, ExtractFix, BigVul+CVEFixes, Zero-Day, and PrimeVul) for exemplar retrieval: when testing on one dataset, samples from the other datasets are used as the pool for the exemplar selection.

Naive CoT (based Few-Shot Learning). This strategy extends standard few-shot learning by incorporating a basic form of Chain-of-Thought (CoT) [92]. In addition to the question, code, and final answer, each exemplar includes a line-by-line analysis of the code, mainly describing its functionality semantics without focusing on security-relevant code. As illustrated in Figure 1, this analysis proceeds sequentially through the code, explaining what each significant line or block of code does. The rationale behind this 'naive' approach is to evaluate whether a straightforward, generic CoT, without deep vulnerability-specific guidance, can prompt LLMs to generate more structured and accurate reasoning. This is to evaluate whether LLMs perform better on vulnerability analysis by giving exemplars with line-by-line code semantics analysis. We used the same number and source of exemplars as in standard few-shot learning.

To minimize subjectivity and ensure replicability, the "line-by-line analysis" used in Naive CoT follows a fixed and constrained construction procedure rather than free-form explanation. Specifically, each Naive CoT exemplar is generated by applying the following deterministic rules:

Standard Few-Shot (Learning). In each prompt, we provide exemplars which include the questions, code samples, and the final answers without any reasoning steps. This is to assess whether a simple instantiation of few-shot learning can help LLMs with vulnerability analysis. In this study, we manually construct exemplars with the same CWEs as the testing samples. In particular, for each of the 5 CWEs in our datasets, we provide two exemplars. This results in a total of 10 exemplars for prompting—we target 10 as in [92]. We curate the exemplar code samples using the demonstrative examples (with appropriate modification) on the official CWE website [1]. For Task 1, we also provide 10 non-vulnerable exemplars to balance.

Retrieval-Augmented Generation (RAG). To address concerns about insufficient code contexts and to explore more advanced few-shot techniques, we incorporate Retrieval Augmented Generation (RAG). RAG aims to provide the LLM with highly relevant in-context examples by dynamically retrieving them based on the input query, thereby potentially mitigat-

- (1) Sequential Coverage Rule: Each statement or contiguous block of statements in the function body is described in source-code order without reordering or omission.
- (2) Syntactic Description Rule: Each description describes the syntactic functionality (e.g., variable initialization, function calls, loop iteration, data assignment) derived directly from the source code, and tries to identify the abnormal behaviors.
- (3) No Dependency Inference Rule: Data-flow or control-flow relationships are not inferred unless explicitly encoded in the syntax (e.g., loop conditions), and no backward or forward reasoning is performed.
- (4) Under these constraints, Naive CoT serves as a baseline that reflects generic program summarization. This design allows us to evaluate whether generic chain-of-thought structure alone can improve LLM performance.

```
// Vulnerability Detection
Q: Does the following code have exploitable vulnerabilities? Please
check whether the vulnerability, if any, can be practically exploited
to cause real consequences.

{Code to analyze}

A: Step 1: Vulnerability semantics approximation:
The function has an input "hello". The number of offers "hello" can store
is assigned to "num" at line 5. At lines 7-8, The offers from "hello" are
assigned to a stack-based buffer "offerd" which is defined at line 6.
Therefore, the vulnerability semantics is:
1 void cycle0(struct st *hello)
5 int num = hello->getOfferNum();
6 int offerd[max_offerd + 1];
7 for (int i = 0; i < num; i++)
8   offerd[i] = hello->getOffer(i);
Step 2: Vulnerability semantics reasoning:
{Vulnerability semantics description}
The security-related behavior may cause crashes or exploitable conditions.
Conclusion: the code is vulnerable.
```

Fig. 3. VSP vulnerability-detection exemplar (for the example in Fig. 2).

et al. [75] propose a prompting strategy that removes or comments out vulnerable code fragments and lets LLMs complete the code. They explored several options and found that the "s.2" version worked most effectively for repairing vulnerabilities in real-world projects, which simply removes the vulnerable code and uses a comment `/*fix [error name] bug*/` to prompt the LLMs for the task. In our study, `[error name]` is the CWE ID of the vulnerability. We refer to this prompting strategy as *zero-shot compl.* and only consider it in Task 3.

```
// Vulnerability Classification
Q: The following code has been confirmed as vulnerable,
which CWE does it have?

{Code to analyze}

A: Step 1: Vulnerability semantics approximation:
..... // same as Task 1
Step 2: Vulnerability semantics reasoning:
{Vulnerability semantics description}
The security-related behavior may cause stack-based buffer
overflow (CWE-121).
Conclusion: the code has a CWE-121 vulnerability.
```

Fig. 4. VSP vulnerability-classification exemplar (for the example in Fig. 2).

performance of LLMs with existing prompting strategies. Thus, we improve Naive CoT into a vulnerability-specific prompting strategy—*Vulnerability-Semantics-guided Prompting (VSP)*. It hones in on *vulnerability semantics*, the pivotal element of effective vulnerability analysis. This improvement is underpinned by three key insights: (1) while a program may encompass numerous lines of code, only a small fraction may be susceptible to vulnerabilities; (2) mainstream LLMs are based on Transformer [88], which is known to exhibit diminished performance when handling extensive text [69], and (3) although LLMs may be good at language understanding tasks, they

We do not optimize or iteratively refine prompt phrasing at the lexical level (e.g., specific adjectives or sentence structures), as doing so would introduce uncontrolled variability and confound comparisons across prompting strategies. Instead, we fix prompt wording once per strategy and treat the presence or absence of reasoning structure and semantic guidance as the primary independent variable. This design choice aligns with our goal of evaluating prompting strategies as methodological categories rather than handcrafted prompt instances.

Zero-Shot (Code) Completion. Specifically for vulnerability analysis (repair in particular), Pearce

While more complex prompting strategies exist, such as Tree of Thoughts [99], they often involve multiple LLM calls, complex parsing of intermediate outputs, or integration with external tools/agents. Our study specifically focuses on direct prompt engineering to derive actionable insights that are broadly applicable to a wide range of users without requiring significant computational resources or specialized model training expertise. Therefore, we exclude these strategies.

2.4.2 Vulnerability Semantics guided Prompting. After preliminary evaluation, we observed undesirable per-

may not be as good at reasoning complex logic, including code analysis [61]—thus, concentrating on vulnerability semantics aids in curtailing the length of exemplars/prompts/answers and focusing on the core reasoning of vulnerabilities, mitigating potential LLM limitations.

```
// Vulnerability Repair
Q: The following code has a CVE-125 out-of-bound read vulnerability at
line 8. Please provide a patch, only showing the code changes needed
rather than the entire patched code, to conform to the conventional
patch format.

(Code to analyze)
A: Step 1: Vulnerability semantics approximation:
// same as Task 1
Step 2: Vulnerability semantics reasoning:
{vulnerability semantics description}
The security-related behavior may cause stack-based buffer overflow
(CVE-125).
Step 3: Repair strategy identification and application:
The vulnerability occurs because the loop does not ensure that 'i' is
within the bounds of the allocated array size 'max_offered + 1'.
Proper bounds checking should be added to ensure 'i' does not exceed
max_offered + 1. Thus, the patch is:
- for (int i = 0; i < num; i++)
+ for (int i = 0; i < num && i < max_offered + 1; i++)
```

Fig. 5. VSP vulnerability-repair exemplar (for the example in Fig. 2).

Vulnerability Semantics. We define vulnerability semantics of a program as its *code semantics that makes it vulnerable*, which can be approximated by (a) sensitive (i.e., likely vulnerable) statements (e.g., those performing memory operations, numeric computation, and pointer manipulation) and (b) their relevant contextual statements (e.g., those enabling control reachability to the operations/computation). This can be computed with a union of a forward slice and a backward slice. Figure 2 illustrates the vulnerability semantics of a code example. To approximate the semantics, we start with the inputs of the program (i.e., the parameter at line 1). Then, we identify the statements that have (forward) data and control dependencies on the input parameter (lines 5, 7) until a sensitive statement (line 8). Finally, we include line 6 which has backward data dependency of the buffer offered used at line 8. Therefore, lines 1 and 5-8 constitute the (approximate) vulnerability semantics.

Note that the vulnerability semantics is either manually identified (for exemplars) or approximated by LLMs during their reasoning (for testing samples), without any pre-extracted data/control-flow information. The manually identified vulnerability semantics in the exemplars is for demonstrating to the LLM the reasoning (i.e., how to identify and focus on vulnerability-relevant behaviors) we expect it to perform. During the testing phase, LLMs are expected to generate their own reasoning and predictions based on the provided code and the patterns learned from the VSP exemplars.

VSP. Guided by vulnerability semantics, we improve Naive CoT to realize VSP. Specifically, each VSP prompt includes two parts. The first part consists of exemplars that provide examples of the reasoning steps for the model to follow CoT. As shown in Figures 3, 4, and 5, each exemplar consists of a question for the task, a code sample, and the answer for the question which includes the reasoning steps. Importantly, these steps describe vulnerability semantics by focusing on code relationships that are relevant to explaining the vulnerable behavior, rather than on general functionality.

Such reasoning steps not only help LLMs focus on the important code fragments for vulnerability analysis but also guide LLMs to focus on the vulnerability behaviors. We provide the same number of exemplars as in standard few-shot learning as discussed earlier—the example code samples are the same. Following these exemplars, the second part of a VSP prompt is the testing sample and the same question as in the exemplars. With the whole VSP prompt as input, LLMs should output the vulnerability analysis results following the vulnerability-semantics-guided reasoning steps in the exemplars. While we provide exemplars with the same CWEs as the testing samples, we note that VSP is not limited to the specific CWEs we evaluate—it can be easily adapted to any CWE with demonstrative examples on the official CWE website [1].

We note that VSP does not enable LLMs to perform sound or complete data/control-flow analysis comparable to traditional program analysis techniques. Instead, VSP is a prompt-level strategy that biases the model’s reasoning focus toward vulnerability-relevant code relationships, as illustrated in the exemplars. The intent is to reduce attention to general functionality and encourage consideration of code semantics that are commonly associated with vulnerability manifestation, without assuming that the resulting reasoning corresponds to precise or exhaustive dependency analysis.

While both Naive CoT and VSP adopt a chain-of-thought structure, they differ fundamentally in their reasoning objectives. Naive CoT enforces a functionality-driven summarization of code behavior, whereas VSP explicitly guides models to identify and reason over vulnerability-relevant semantics. As a result, Naive CoT is intentionally limited to evaluating whether functionality-based reasoning alone is sufficient, while VSP evaluates whether task-specific semantic guidance is necessary for effective vulnerability analysis.

Table 2. Traditional analysis approaches compared

Class	Tool	Year	Technique	Tasks
<i>Code analysis based</i>	CodeQL [3]	2024	static analysis/code querying	1,2
	FlawFinder [2]	2022	static analysis/pattern matching	1,2
	CppCheck [59]	2024	static analysis/symbolic execution	1,2
	ExtractFix [32]	2021	dynamic analysis/symbolic execution	3
	VulnFix [101]	2022	dynamic analysis/inductive inference	3
<i>DL based</i>	VulChecker [60]	2023	graph neural network/training	1,2
	LineVul [28]	2022	Transformer (BERT)/fine-tuning (FT)	1
	VulRepair [29]	2022	Transformer (CodeT5)/pre-training+FT	3
	Devign [105]	2019	(gated) graph neural network/training	1

Zero-Shot VSP. To assess whether exemplars are essential for improving LLMs’ capabilities, we also consider zero-shot VSP, a VSP variant where we replace the exemplars with a description of VSP steps in the prompt: “(1) approximate the code semantics that make it vulnerable and (2) reason about its underlying code dependencies to analyze vulnerabilities.”

2.5 Decoding Strategies and System Prompts

In this study, we investigate how the configuration of the LLMs—specifically decoding strategies and system prompts—impacts their effectiveness in vulnerability analysis.

Decoding Strategies. To assess the sensitivity of LLMs to generation parameters, we examine two key decoding dimensions: temperature and Top-P (nucleus sampling). We prioritize these two parameters as they are the most ubiquitous hyperparameters governing the trade-off between output diversity and determinism, directly influencing whether the model adheres to the most probable reasoning paths or explores alternative possibilities. Temperature controls the stochasticity of the model’s output, where 0 ensures deterministic results and 1 allows for maximum diversity. Top-P restricts token generation to the top subset of tokens with a cumulative probability of P. We observe how constraining the probability space impacts the accuracy of vulnerability semantics analysis.

System Prompts. We investigate the impact of defining a specific persona for the LLM through system-level instructions. We compare three distinct settings: No System Prompt, which provides the VSP with input directly without behavioral constraints; Helpful Assistant, which applies the generic instruction “You are a helpful assistant”; and Security Expert, which primes the model with the persona “You are an expert in software security and vulnerability analysis.” We selected these specific prompts to bracket the potential impact of persona adoption: the “Helpful Assistant” serves as a baseline representing the default alignment behavior of modern LLMs, while the “Security Expert” tests the hypothesis that domain-specific priming activates latent security knowledge and enforces stricter reasoning standards.

2.6 Traditional Approaches Compared

To assess where LLMs stand among extant vulnerability analysis solutions, we consider nine traditional approaches for the three tasks, as summarized in Table 2. The year shown is that of the latest release of respective tools accessible by Sep. 2024. The choices are justified by their coverage of (1) the latest SOTA, popular, and/or most effective tools/techniques in respective classes for the tasks they are capable of, (2) both static and dynamic analysis techniques, and (3) various deep learning (DL) models and training schemes. For instance, CodeQL, CppCheck, and FlawFinder are among the best static application security testing (SAST) tools [56] widely used in practice, while VulChecker represents the most recent advance in DL-based vulnerability detection/classification. We used their default or most widely used configuration to test their performance. Specifically, for ExtractFix and VulnFix, we used their default setting that allowed us to reproduce the results reported in their respective original papers. For CppCheck and FlawFinder, we also use their default setting without special arguments/options,

which enabled the full range of memory error checking. For CodeQL, we use the security query pack for C/C++ to detect/classify the vulnerabilities. For DL-based models, to ensure a fair comparison and mitigate potential performance degradation due to distribution shifts, we evaluated them in two settings: (1) using their originally provided pre-trained models, and (2) re-training them on the large-scale DiverseVul dataset. This ensures that the DL-based baselines are optimized for the vulnerability analysis tasks on a dataset distribution comparable to what the fine-tuned LLMs experienced, providing a robust "best-case" baseline for traditional DL approaches.

2.7 Implementation and Experiment Settings

For Gemini, Claude3, GPT-3.5, and GPT-4, we subscribe to the APIs from respective companies to process the input and generate the results. For Llama2, Falcon, CodeLlama, Qwen2.5-Coder, and DeepSeek-Coder-V2, we download the models from HuggingFace [93], deploy locally, and use their APIs to conduct the experiments. For fine-tuning the Llama2-13b model, we utilized the Unsloth framework [35] to perform efficient instruction tuning. We employed the standard prompting strategy (as described in §2.4.1) to format the input and output for the fine-tuning samples. We perform our experiments on a workstation with an AMD Ryzen Threadripper Pro 5595WX (4.5GHz) CPU with 64 Cores, four Nvidia GeForce RTX A6000 GPUs, and 512GB memory.

```
1 static void esp_do_dma(ESPState *s){
2     uint32_t len;
3     int to_device;
4     if (!s)
5         return;
6     len = s->dma_left;
7     if (s->do_cmd){
8         trace_esp_do_dma(s->cmdlen, len);
9         s->dma_memory_read(s->dma_opaque, &s->cmdbuf[s->cmdlen], len);
10        return;
11    }
12    return; }
```

Answer with VSP prompt:

A: Step 1: Find vulnerability semantics:
The function has an input "s". The function reads from "s" and writes to "s->cmdbuf". Therefore, the vulnerability semantics is:
1 static void esp_do_dma(ESPState *s)
6 len = s->dma_left;
8 trace_esp_do_dma(s->cmdlen, len);
9 s->dma_memory_read(s->dma_opaque, &s->cmdbuf[s->cmdlen], len);
Step 2: Vulnerability semantics analysis:
At line 6, the value of "s->dma_left" is assigned to "len".
At lines 8-9, the function reads from "s->dma_memory_read" and writes to "&s->cmdbuf[s->cmdlen]". However, the size of "&s->cmdbuf[s->cmdlen]" may be less than "len", causing a heap-based buffer overflow (CWE-122).
Conclusion: the code has an exploitable vulnerability.

Answer with Standard Prompt:

Based on the provided code, it is difficult to determine if there are any exploitable vulnerabilities or if the inputs can be maliciously used to crash the program. The code appears to be a function that performs a DMA (Direct Memory Access) operation, but without additional context or information about the input validation and usage, it is not possible to definitively determine if there are any vulnerabilities.

Answer with Naive CoT Prompt:

A: 1. The code defines a static function named "esp_do_dma" that takes a pointer to an "ESPState" struct named "s" as a parameter.
.....
11. The function returns. This line is not necessary as it is unreachable code.
This code does not have any obvious vulnerability as it is a simple function that performs some operations based on the provided "ESPState" struct. However, without the implementation details of the "ESPState" struct and the "dma_memory_read" function, it is not possible to determine if there are any exploitable vulnerabilities.
Conclusion: the code does not have an exploitable vulnerability based on the provided information.

Fig. 6. GPT-3.5 detects a CWE-122 vulnerability with VSP but not with Standard Prompting or naive CoT.

and their optimized performance when adapted to the latest vulnerability datasets.

To ensure the reproducibility and consistency of our primary evaluations (RQ1, RQ2, and RQ4–RQ7), we utilized a fixed default configuration for all LLMs. Specifically, we set the temperature to 0 to minimize stochasticity and maximize the determinism of the outputs. We set the Top-P (nucleus sampling) to 1, allowing the model to consider the full vocabulary probability distribution, though the generation is effectively constrained by the zero temperature. Furthermore, regarding system instructions, we employed the No System Prompt setting as the default; this means the models received only the task-specific prompts (as detailed in §2.4) without any additional persona definitions or behavioral instructions (e.g., "You are a helpful assistant"). We only deviate from these fixed defaults in RQ3, where we explicitly vary these parameters to assess the sensitivity of LLMs to decoding strategies and system prompts.

For the traditional baseline techniques, we first follow their instructions to reproduce the results reported in their papers. Then, we use the same configurations to evaluate them against our datasets. For DL-based techniques, we evaluate them in two settings to ensure a comprehensive comparison: using their originally provided pre-trained models, and using models fine-tuned on the large-scale DiverseVul dataset. This allows us to assess both their off-the-shelf capabilities

Table 3. Vulnerability Detection (Task 1) effectiveness of LLMs with different prompting strategies. SP: Standard Prompting; SF: Standard Few-Shot; NC: Naive CoT; RAG: Retrieval-Augmented Generation; ZVSP: Zero-Shot VSP; VSP: Vulnerability-Semantics-guided Prompting. QC-2.5: Qwen2.5-Coder; DSC-V2: DeepSeek-Coder-V2; FT-Llama2: Fine-Tuned Llama2-13b

LLM	PS	VulChecker					BigVul+CVEFixes					Zero-Day					PrimeVul-Subset					PrimeVul-Full				
		Rec	Prec	F1	MCC	AuPRC	Rec	Prec	F1	MCC	AuPRC	Rec	Prec	F1	MCC	AuPRC	Rec	Prec	F1	MCC	AuPRC	Rec	Prec	F1	MCC	AuPRC
Gemini	SP	36.07%	16.42%	22.56%	0.1306	0.2908	31.82%	20.19%	24.71%	0.2158	0.2732	16.50%	44.73%	34.71%	-0.0501	0.5149	28.13%	27.11%	0.2560	0.2868	0.2868	60.69%	23.59%	33.98%	0.3526	0.4266
	SF	55.42%	25.84%	35.25%	0.2893	0.4261	54.47%	43.31%	48.25%	0.4626	0.4977	28.15%	45.31%	34.73%	-0.0630	0.5469	46.01%	38.15%	41.72%	0.4016	0.4288	54.10%	21.52%	30.79%	0.3138	0.3842
	NC	9.21%	9.59%	9.39%	0.0077	0.1343	11.01%	23.00%	15.29%	0.1444	0.1972	3.92%	26.67%	6.84%	-0.1314	0.3932	8.05%	20.42%	11.54%	0.1136	0.1559	49.36%	19.72%	28.19%	0.2832	0.3522
	RAG	27.05%	56.09%	36.51%	0.3511	0.4481	25.88%	16.42%	20.09%	0.1660	0.2258	22.33%	57.50%	32.17%	0.0736	0.5933	25.09%	43.34%	31.78%	0.3161	0.3532	11.65%	46.15%	18.60%	0.2222	0.3008
	ZVSP	81.93%	10.88%	19.21%	0.1002	0.4721	81.03%	15.38%	25.85%	0.3023	0.4857	32.03%	54.09%	40.24%	0.0531	0.6005	65.00%	26.78%	37.93%	0.3938	0.4641	52.82%	18.49%	27.40%	0.2822	0.3629
	VSP	80.00%	26.12%	39.38%	0.3705	0.5395	74.19%	40.05%	52.02%	0.5214	0.5762	33.33%	54.55%	41.38%	0.0604	0.6061	62.51%	40.24%	48.96%	0.4849	0.5193	63.75%	31.00%	41.72%	0.4237	0.4786
Claude3	SP	71.43%	14.15%	23.62%	0.1672	0.4406	63.63%	20.00%	30.43%	0.3136	0.4252	37.86%	50.65%	43.33%	0.0100	0.5979	57.64%	28.27%	37.93%	0.3811	0.4358	17.18%	19.31%	18.18%	0.1611	0.1935
	SF	53.01%	7.43%	13.04%	-0.0666	0.3230	33.18%	8.25%	13.21%	0.0979	0.2200	29.41%	56.60%	38.71%	0.0782	0.6065	38.53%	24.09%	29.65%	0.2808	0.3222	14.11%	16.67%	15.28%	0.1321	0.1653
	NC	8.95%	12.82%	10.53%	0.0356	0.1492	10.62%	60.00%	18.05%	0.2417	0.3703	24.21%	52.27%	33.09%	0.0249	0.5719	14.59%	41.70%	21.61%	0.2351	0.2940	14.72%	20.87%	17.27%	0.1565	0.1893
	RAG	10.64%	35.71%	16.39%	0.1555	0.2714	21.33%	58.50%	31.26%	0.3389	0.4143	3.64%	33.33%	6.56%	-0.0801	0.4258	11.87%	42.52%	18.56%	0.2142	0.2849	30.67%	25.00%	27.55%	0.2550	0.2876
	ZVSP	57.65%	10.23%	17.38%	0.0477	0.3582	75.30%	19.75%	31.29%	0.3428	0.4800	11.76%	46.15%	18.75%	-0.0294	0.5102	48.24%	25.38%	33.26%	0.3258	0.3757	13.50%	27.50%	18.11%	0.1775	0.2165
	VSP	57.57%	45.24%	50.67%	0.4563	0.5329	51.28%	76.92%	61.54%	0.6164	0.6504	32.04%	70.21%	44.00%	0.2198	0.6812	46.96%	64.12%	54.22%	0.5384	0.5633	49.08%	19.46%	27.87%	0.2800	0.3495
GPT-3.5	SP	4.71%	4.44%	4.57%	-0.0502	0.0680	2.14%	80.00%	4.16%	0.1271	0.4295	2.91%	60.00%	5.56%	0.0315	0.5573	3.25%	48.15%	6.09%	0.1200	0.2713	3.11%	6.85%	4.27%	0.0288	0.0627
	SF	32.94%	84.85%	47.45%	0.5047	0.6187	16.90%	95.18%	28.70%	0.3938	0.5764	4.85%	55.56%	8.93%	0.0238	0.5399	18.23%	78.53%	29.59%	0.3717	0.4959	8.70%	16.87%	11.48%	0.1041	0.1400
	NC	23.53%	28.17%	25.64%	0.1920	0.2924	17.29%	48.20%	25.45%	0.2728	0.3434	23.81%	83.33%	37.04%	0.0721	0.7262	21.54%	53.23%	30.67%	0.3279	0.3855	13.04%	36.84%	19.27%	0.2071	0.2610
	RAG	85.71%	24.00%	37.50%	0.3623	0.5549	16.00%	54.55%	24.74%	0.2816	0.3689	27.18%	57.14%	36.84%	0.0798	0.6037	42.97%	45.23%	44.07%	0.4260	0.4494	32.65%	22.61%	26.72%	0.2478	0.2853
	ZVSP	47.05%	15.15%	22.92%	0.1362	0.3345	48.56%	15.82%	23.87%	0.2277	0.3318	9.71%	66.67%	16.95%	0.0934	0.6076	35.11%	32.55%	33.78%	0.3192	0.3479	40.37%	18.95%	25.79%	0.2481	0.3045
	VSP	50.79%	58.18%	54.23%	0.5025	0.5667	33.08%	72.03%	45.34%	0.4754	0.5384	33.01%	59.65%	42.50%	0.1194	0.6308	38.96%	63.29%	48.23%	0.4861	0.5240	39.88%	26.32%	31.71%	0.3013	0.3390
GPT-4	SP	6.67%	16.67%	9.52%	0.0526	0.1581	6.81%	25.00%	10.71%	0.1132	0.1770	26.31%	65.22%	37.50%	0.0530	0.6419	13.26%	35.63%	19.33%	0.2049	0.2573	19.02%	19.25%	19.14%	0.1694	0.2021
	SF	23.33%	63.63%	34.14%	0.3531	0.4688	11.36%	83.33%	20.00%	0.3001	0.4905	21.05%	83.33%	33.61%	0.0535	0.7193	18.58%	76.76%	29.92%	0.3707	0.4887	16.59%	86.50%	27.84%	0.3731	0.5265
	NC	60.00%	50.00%	54.55%	0.4993	0.5677	40.91%	60.00%	48.64%	0.4792	0.5159	34.95%	63.16%	45.00%	0.0628	0.6532	45.29%	57.72%	50.75%	0.4996	0.5281	7.36%	17.65%	10.39%	0.0987	0.1374
	RAG	82.98%	25.00%	38.42%	0.3664	0.5474	35.92%	62.16%	45.14%	0.4572	0.5027	42.72%	50.00%	46.07%	0.0000	0.6068	53.87%	45.72%	49.46%	0.4814	0.5048	39.26%	22.07%	28.26%	0.2688	0.3147
	ZVSP	36.67%	5.16%	9.05%	-0.1706	0.2372	43.18%	9.13%	15.79%	0.1295	0.2725	32.25%	54.59%	37.50%	-0.0814	0.5536	37.37%	19.63%	25.74%	0.2435	0.2942	31.14%	16.36%	21.45%	0.1966	0.2466
	VSP	70.00%	56.75%	62.68%	0.5903	0.6471	56.81%	49.01%	52.63%	0.5073	0.5374	44.04%	54.09%	48.70%	0.0822	0.6564	60.39%	49.95%	54.68%	0.5357	0.5576	39.13%	32.47%	35.49%	0.3372	0.3661
Llama2-7b	SP	16.67%	2.42%	4.22%	-0.2845	0.1324	46.87%	6.61%	11.58%	0.0879	0.2776	43.14%	50.00%	46.31%	0.0000	0.6079	35.56%	19.68%	25.33%	0.2376	0.2857	19.06%	10.85%	13.83%	0.1127	0.1603
	SF	28.13%	11.53%	16.36%	0.0491	0.2302	25.33%	10.05%	14.39%	0.1056	0.1913	22.54%	63.89%	33.33%	0.1286	0.6258	25.33%	28.49%	26.52%	0.2498	0.2801	16.54%	7.31%	10.14%	0.0731	0.1303
	NC	12.50%	7.01%	8.98%	-0.0284	0.1364	10.00%	6.25%	7.69%	0.0319	0.0986	45.09%	48.42%	46.70%	-0.0295	0.6048	22.53%	20.56%	21.50%	0.1928	0.2689	23.98%	14.06%	17.72%	0.1544	0.2003
	RAG	33.33%	4.17%	7.41%	-0.2588	0.2171	44.68%	15.44%	22.95%	0.2133	0.3113	22.53%	29.17%	25.34%	-0.3304	0.4522	33.51%	16.26%	21.90%	0.2036	0.2587	7.72%	13.29%	9.77%	0.0828	0.1173
	ZVSP	31.25%	4.41%	7.72%	-0.2042	0.2088	53.13%	8.13%	14.11%	0.1289	0.3153	30.39%	48.43%	37.35%	-0.0212	0.5681	38.26%	20.32%	26.54%	0.2520	0.3020	24.46%	10.06%	14.57%	0.1261	0.1923
	VSP	42.85%	37.50%	39.99%	0.3382	0.4271	53.12%	44.74%	48.57%	0.4651	0.4983	38.23%	60.93%	46.98%	0.1478	0.6502	44.73%	47.72%	46.18%	0.4478	0.4704	39.96%	12.31%	18.82%	0.1844	0.2693
Llama2-13b	SP	54.12%	8.11%	14.11%	-0.0323	0.3315	42.08%	17.37%	24.58%	0.2254	0.3084	47.06%	47.06%	47.06%	-0.0588	0.6030	47.75%	24.18%	32.10%	0.3149	0.3674	28.44%	11.85%	16.73%	0.1492	0.2110
	SF	64.71%	23.50%	34.48%	0.2925	0.4567	71.13%	28.38%	40.56%	0.4166	0.5031	21.56%	55.00%	30.99%	0.0494	0.5789	52.47%	35.63%	42.44%	0.4137	0.4475	34.56%	11.39%	17.13%	0.1608	0.2385
	NC	14.28%	14.12%	14.20%	0.0579	0.1800	14.18%	25.00%	18.09%	0.1642	0.2124	9.50%	50.00%	16.39%	0.0000	0.5245	12.75%	29.71%	17.85%	0.1807	0.2252	58.37%	12.23%	20.23%	0.2268	0.3585
	RAG	52.94%	12.30%	19.96%	0.0946	0.3471	48.45%	21.61%	29.89%	0.2837	0.3602	42.43%	57.56%	32.27%	0.0744	0.5939	41.27%	30.49%	35.07%	0.3343	0.3675	9.89%	9.27%	9.57%	0.0702	0.1078
	ZVSP	36.47%	5.96%	10.24%	-0.1116	0.2403	22.53%	7.25%	10.96%	0.0651	0.1638	17.64%	47.37%	25.71%	-0.0252	0.5310	25.55%	20.19%	22.56%	0.2033	0.2397	54.26%	11.62%	19.14%	0.2097	0.3355
	VSP	54.12%	32.17%	40.35%	0.3432	0.4518	51.63%	50.48%	51.04%	0.4907	0.5199	39.22%	63.49%	48.48%	0.1404	0.6655	48.32%	48.71%	48.52%	0.4712	0.4928	57.87%	12.92%	21.13%	0.2345	0.3595
Falcon	SP	54.12%	7.31%	12.88%	-0.0760	0.3275	61.90%	9.56%	15.56%	0.1710	0.3646	33.33%	46.15%	38.71%	-0.0579	0.5641	49.75%	21.01%	29.55%	0.2957	0.3614	12.15%	6.99%	8.87%	0.0592	0.1074
	SF	37.08%	37.93%	31.74	0.4066	0.4666	49.01%	8.29%	14.21%	0.1262	0.2992	29.41%	56.60%	38.71%	0.0782	0.6065	38.98%	33.99%	36.44%	0.3467	0.3753	11.76%	6.92%	8.39%	0.0539	0.1032
	NC	75.29%	15.27%	25.39%	0.1984	0.4638	54.95%	13.62%	21.82%	0.2177	0.3515	24.21%	52.27%	33.09%	0.0249	0.5719	51.48%	27.05%	35.47%	0.3501	0.3999	10.00%	9.46%	9.72%	0.0719	0.1093
	RAG	16.67%	6.54%	9.40%	-0.0443	0.1530	24.82%	16.06%	19.50%	0.1596	0.2189	26.67%	57.14%	36.36%	0.0788	0.6024	22.72%	26.58%	24.50%	0.2267	0.2579	4.26%	5.71%	4.78%	0.0207	0.0626
	ZVSP	41.18%	9.07%	14.86%	0.0057	0.2773	43.77%	8.18%	13.78%	0.1146	0.2706	11.76%	46.15%	18.75%	-0.0294	0.5102	32.24%	21.13%	25.33%	0.2361	0.2769	11.22%	7.28%	8.84%	0.0594	0.1044
	VSP	63.53%	27.41%	38.29%	0.3317	0.4709	44.77%	36.83%	40.41%	0.3798	0.4186	35.29%	43.37%	38.92%	0.0898	0.5551	47.86									

Table 4. Statistical significance and size of differences in effectiveness between different prompting strategies. SP: Standard Prompting; SF: Standard Few-Shot; NC: Naive CoT; ZSC: Zero-Shot Completion; RAG: Retrieval-Augmented Generation; ZVSP: Zero-Shot VSP; VSP: Vulnerability-Semantics-guided Prompting. ---: Not applicable; ZSC cannot be applied to detection or classification tasks

Compared Strategies	Detection						Classification						Repair	
	p-value			Effect Size			p-value			Effect Size			p-value	Effect Size
	F1	MCC	AuPRC	F1	MCC	AuPRC	micF1	MCC	micAu	micF1	MCC	micAu	Correct	Correct
SP:SF	0.3414	0.0158	0.0971	-0.1104	-0.1775	-0.1379	0.1556	0.0385	0.9900	-0.0651	-0.0823	-0.0413	0.0059	-0.2124
SP:NC	0.3745	0.5352	0.3269	0.0413	-0.0542	0.0188	0.0045	0.0023	0.0641	-0.1540	-0.1825	-0.1554	0.0003	-0.2336
SP:ZSC	-	-	-	-	-	-	-	-	-	-	-	-	0.0306	0.0680
SP:RAG	0.6034	0.3459	0.7566	-0.0334	-0.0972	-0.0083	0.9554	0.4438	0.2212	-0.0298	0.0294	0.0628	0.0009	-0.2136
SP:ZVSP	0.8397	0.3745	0.3524	-0.0311	-0.0486	-0.0760	0.0002	0.0298	0.2611	-0.1002	-0.0767	-0.0952	0.0037	-0.1556
SP:VSP	0.0000	0.0000	0.0000	-0.6413	-0.5577	-0.4856	0.0000	0.0000	0.0000	-0.4436	-0.4655	-0.4387	0.0000	-0.4628
SF:NC	0.1496	0.1381	0.0703	0.1094	0.1160	0.1458	0.0859	0.0458	0.0081	-0.0850	-0.1243	-0.1481	0.1577	-0.0704
SF:ZSC	-	-	-	-	-	-	-	-	-	-	-	-	0.0003	0.3060
SF:RAG	0.3836	0.1114	0.0125	0.0618	0.0797	0.1207	0.1941	0.0282	0.1999	0.0288	0.1127	0.1104	0.7881	0.0220
SF:ZVSP	0.4868	0.1828	0.6329	0.0721	0.1369	0.0668	0.4559	0.8406	0.1720	-0.0284	0.0000	-0.0740	0.5932	0.0124
SF:VSP	0.0000	0.0000	0.0000	-0.5587	-0.4050	-0.3256	0.0000	0.0000	0.0000	-0.3904	-0.4165	-0.4093	0.0000	-0.3252
NC:ZSC	-	-	-	-	-	-	-	-	-	-	-	-	0.0000	0.3372
NC:RAG	0.3438	0.9466	0.5463	-0.0638	-0.0486	-0.0393	0.0104	0.0003	0.0026	0.1084	0.2288	0.2483	0.2005	0.0648
NC:ZVSP	0.5243	0.9399	0.1643	-0.1071	-0.0043	-0.1306	0.5297	0.2598	0.4433	0.0588	0.1051	0.0400	0.0174	0.0952
NC:VSP	0.0000	0.0000	0.0000	-0.6066	-0.5015	-0.4744	0.0000	0.0000	0.0000	-0.3150	-0.3236	-0.2820	0.0000	-0.2496
RAG:ZSC	-	-	-	-	-	-	-	-	-	-	-	-	0.0004	-0.2888
RAG:ZVSP	0.5575	0.7061	0.2687	0.0281	0.0631	-0.0565	0.0370	0.0146	0.0071	-0.0598	-0.1293	-0.1719	0.0001	0.0280
RAG:VSP	0.0000	0.0000	0.0000	-0.5722	-0.4486	-0.4555	0.0000	0.0000	0.0000	-0.4026	-0.5131	-0.4952	0.0000	-0.3180
ZVSP:ZSC	-	-	-	-	-	-	-	-	-	-	-	-	0.3121	-0.2600
ZVSP:VSP	0.0000	0.0000	0.0000	-0.6605	-0.5213	-0.4592	0.0000	0.0000	0.0000	-0.3577	-0.3997	-0.3345	0.0000	-0.3516
ZSC:VSP	-	-	-	-	-	-	-	-	-	-	-	-	0.0000	-0.5712

predicted and actual classifications, which is particularly useful for imbalanced datasets, while AuPRC evaluates the trade-off between precision and recall across different decision thresholds.

We first look at LLMs' performance with existing prompting strategies. As shown in Table 3, the performance varies widely. While F1 scores range up to 54.55%, the MCC values reveal deeper reliability issues. For instance, with Standard Prompting, Gemini and Llama2-13b exhibit negative MCCs on the Zero-Day dataset (-0.0501 and -0.0588 respectively) and VulChecker (-0.2845 for Llama2-7b), indicating their predictions are often worse than or equivalent to random guessing. Similarly, AuPRC values are often low for standard prompting (e.g., 0.0880 for GPT-3.5 on VulChecker), suggesting the models struggle to rank vulnerable samples correctly without guidance.

Over models of different vendors, there is no consistent relevancy between the performance and model size, age, or capacity (max #tokens). For example, compared to Falcon, Llama2-7b has the same size (7B parameters) and even greater capacity (up to 4096 tokens) but achieved lower performance in some cases (e.g., 16.36% versus 37.93% F1 against the VulChecker dataset, with standard few-shot). Another example is that Gemini is newer than GPT-3.5 (released date Dec. 2023 versus June 2023), but its best F1 score against VulChecker (36.51%) is lower than the one from GPT-3.5 (47.45%). Indeed, there is no model that works the best with all strategies and datasets. Newer/larger models do not necessarily outperform older/smaller ones across all scenarios.

Between the models from the same vendors such as Llama2 and GPT, larger/newer models (i.e., Llama2-13b and GPT-4) consistently achieve higher best F1 than the smaller/older ones. For instance, Llama2-13b's best F1s are 34.48% (VulChecker), 40.56% (BigVul+CVEFixes), 47.06% (Zero-Day), 42.44% (PrimeVul-Subset), and 20.23% (PrimeVul-Full), compared to Llama2-7b's 16.36%, 22.95%, 46.70%, 26.82%, and 17.72% respectively. Similarly, GPT-4 consistently outperforms GPT-3.5. This trend holds across all datasets.

We also notice that the commercial LLMs (i.e., Gemini, Claude3, GPT-3.5, GPT-4) outperform the open-source LLMs (i.e., Llama2-7b, Llama2-13b, Falcon, CodeLlama, Qwen2.5-Coder, DeepSeek-Coder-V2), where the commercial LLMs achieve 23.62%-54.55% F1 while the open-source LLMs achieve 7.69%-47.06% F1 against the four datasets. This indicates the advantages of commercial LLMs for vulnerability analysis.

```

1 static String HHVM_FUNCTION(bcsb, const String &left,
2   const String &right, int64_t scale) {
3   if (scale < 0) scale = BCG(bc_precision);
4   bc_num first, second, result;
5   bc_init_num(&first);
6   bc_init_num(&second);
7   bc_init_num(&result);
8   php_str2num(&first, (char *)left.data());
9   php_str2num(&second, (char *)right.data());
10  bc_sub(first, second, &result, scale);
11  if (result->n_scale > scale) result->n_scale = scale;
12  String ret(bc_num2str(result), AttachString);
13  bc_free_num(&first);
14  bc_free_num(&second);
15  bc_free_num(&result);
16  return ret; }

```

// Gemini output with VSP
Step 1: Find vulnerability semantics:
The function has String inputs "left" and "right". At lines 9-10, "left" and "right" are converted into numbers "first" and "second" which are declared at line 6. At line 11, "first" and "second" are subtracted into "result" with scale "scale". Therefore, the vulnerability semantics is:
1 static String HHVM_FUNCTION(bcsb, const String &left, const String &right, int64_t scale)
9 php_str2num(&first, (char *)left.data());
10 php_str2num(&second, (char *)right.data());
11 bc_sub(first, second, &result, scale);
Step 2: Vulnerability semantics analysis:
At line 11, "first" and "second" are subtracted into "result" without checking. This may cause an integer overflow (CWE-190) vulnerability.
Conclusion: The code has a CWE-190 vulnerability.

// Gemini output with naive CoT
A: 1. The code defines a static function named "HHVM_FUNCTION(bcsb)" that takes three parameters: a constant reference to an object of type "String" named "left", a constant reference to an object of type "String" named "right", and an int64_t variable named "scale".
2. The function begins.
.....
17. It calls the "bc_free_num" function, passing the address of the "result" object. This line is potentially vulnerable to a CWE-416 vulnerability if "result" was already freed.
18. The function returns the "ret" variable.
19. The function ends.
Conclusion: The code has a CWE-416 vulnerability if any of the "bc_free_num" calls are performed on a previously freed object.

// Llama2-7b output with VSP
A: Step 1: Find vulnerability semantics:
The function has inputs "left", "right", and "scale". The function performs a subtraction operation on two bc_num objects, which may cause a stack-based buffer overflow (CWE-121) vulnerability if the input values are not properly validated.
Step 2: Vulnerability semantics analysis:
At line 9, the "php_str2num" function is called with the input "left" and "right". The function may cause a buffer overflow if the input values are not properly validated.
Conclusion: The code has a CWE-121 vulnerability.

Fig. 7. An example where Gemini correctly classifies a CWE-190 vulnerability with VSP while it fails with naive CoT, while Llama2-7b fails even with VSP.

models like Falcon struggle (0.5641 AuPRC). These metrics reveal that without specific guidance, LLMs often fail to discriminate reliably between vulnerable and non-vulnerable samples, leading to poor predictive stability.

We also evaluate the performance of the fine-tuned Llama2-13b model. With standard prompting, the fine-tuned model generally outperforms its base (i.e., pre-trained) version and other open-source models on the existing datasets. For instance, on the VulChecker dataset, it achieves an F1 score of 34.13%, significantly surpassing the base Llama2-13b (14.11%) and even exceeding GPT-4 with standard prompting (9.52%). This indicates that fine-tuning effectively adapts the model to the task format and distribution of the training data. However, on the Zero-Day dataset, the fine-tuned model with standard prompting (46.30%) does not show a significant advantage over the base model (47.06%), suggesting that fine-tuning on historical data does not automatically guarantee better generalization to unseen vulnerabilities. Furthermore, we observe that the choice of prompting strategy remains crucial for the fine-tuned model. Advanced strategies consistently boost performance over standard

When examining the performance of the code-specific LLMs (CodeLlama, Qwen2.5-Coder, and DeepSeek-Coder-V2) with existing prompting strategies, these models generally exhibit lower F1 scores compared to the leading general-purpose commercial LLMs like GPT-4 and Claude3. For instance, CodeLlama's best F1 across all datasets is 11.13% (RAG on PrimeVul-Subset), Qwen2.5-Coder's is 33.71% (Standard Prompting on BigVul+CVE-Fixes), and DeepSeek-Coder-V2's is 11.56% (Standard Prompting on BigVul+CVEFixes). While their performance varies across datasets and prompting strategies, their peak F1 scores remain considerably below the 54.55% achieved by GPT-4. This suggests that despite their code-centric training, these code-specific LLMs are not yet as effective for complex software vulnerability detection tasks as the more powerful, larger general-purpose LLMs. A potential reason for this could be that while code-specific LLMs are optimized for tasks like code generation or simple bug fixing, their training objectives might not sufficiently emphasize the deep semantic understanding of code that is crucial for identifying subtle security vulnerabilities. General-purpose LLMs, with their broader training on diverse text and code, might develop a more robust understanding of logic and abstract reasoning that translates more effectively to complex vulnerability analysis, even without explicit code-focused pre-training.

The limitations of existing prompting strategies are further highlighted by the MCC and AuPRC metrics. A significant number of models exhibit negative MCC scores, particularly on the Zero-Day dataset, indicating a correlation worse than random guessing. For instance, on the Zero-Day dataset with standard prompting, Gemini scores -0.0501 MCC, DeepSeek-Coder-V2 scores -0.7360, and CodeLlama drops to -0.8944. Even the high-performing GPT-4 only achieves a marginal 0.0530 MCC on the Zero-Day dataset with standard prompting. The AuPRC scores mirror this instability; while some models like GPT-4 maintain a relatively high AuPRC (0.6419 on Zero-Day with standard prompting), smaller

prompting: Standard Few-Shot and RAG improve the F1 score on VulChecker to 44.32% and 46.81%, respectively, while Naive CoT reaches 55.76%.

Among the four existing prompting strategies, standard few-shot achieves the best performance on many cases (e.g., Gemini, GPT-3.5, Llama2-7b, and Llama2-13b achieve the best F1 on the either VulChecker or BigVul+CVE-Fixes datasets with standard few-shot, as marked **red**). Yet naive CoT does not work well on the majority of these LLMs. It only achieves the best F1 for GPT-4 against VulChecker, BigVul+CVEFixes, and PrimeVul-Subset, Llama2-7b and CodeLlama against Zero-Day, as well as Falcon against BigVul+CVEFixes. As Figure 6 shows, with naive CoT, GPT-3.5 just simply scans the code and analyzes the major functionality. It is even worse than standard prompting on Gemini (6.84%-15.29% versus 22.56%-27.61% F1 against the four datasets) and Claude3 (10.53%-33.09% versus 23.62%-43.33% F1 against the four datasets). RAG achieves the best effectiveness for several LLMs against some datasets (e.g., Gemini against VulChecker, Claude3 against BigVul+CVEFixes, GPT-3.5 against Zero-Day and PrimeVul-Subset), indicating that RAG has the potential to mitigate the insufficient context and improve the few-shot learning.

Across different datasets, the effectiveness does not consistently correlate to the LLMs' performance. For instance, seven models (Gemini, Claude3, Llama2-7b, Llama2-13b, CodeLlama, Qwen2.5-Coder, and DeepSeek-Coder-V2) detect vulnerabilities in BigVul+CVEFixes more accurately than in the VulChecker dataset (up to 8.94%-48.25% versus 7.69%-36.51% F1 among different prompting strategies). The other three show the opposite trend (up to 37.93%-54.55% versus 21.82%-48.64% F1 against VulChecker and BigVul+CVEFixes respectively). Notably, performance on existing datasets (VulChecker, BigVul+CVEFixes, PrimeVul-Subset) may be subject to data leakage, as evidenced by the performance drops on the Zero-Day dataset in many cases. (e.g., from 48.25% to 34.73%, 47.45% to 36.84%, 54.55% to 46.07% for Gemini, GPT-3.5, and GPT-4 respectively). The PrimeVul-Subset dataset also presents varied performance, with F1 scores ranging from 3.33% to 50.75%. Its effectiveness is not consistently superior or inferior to the other three datasets. For example, the best F1 of Gemini on PrimeVul-Subset (41.72%) is higher than the ones on VulChecker (36.51%) and Zero-Day (34.73%) but lower than the one on BigVul+CVEFixes (48.25%). This indicates that LLMs can achieve comparable effectiveness on datasets with higher label accuracy.

When evaluated on the PrimeVul-Full dataset, which includes a much wider variety of 62 CWEs compared to the 5 targeted in other datasets, we observe a general decline in performance for most models using existing strategies. For example, Claude3 drops from 37.93% to 18.18% with standard prompting. This indicates that while LLMs may be reasonably effective at detecting common high-profile vulnerabilities (like buffer overflows), they struggle significantly when tasked with identifying a broader spectrum of security flaws without specific guidance. However, Gemini remains relatively robust, achieving the highest F1 of 33.98% with Standard Prompting on this diverse dataset.

Finding 1 (RQ1): In vulnerability detection, large general-purpose LLMs outperform specialized code models. Fine-tuning aids historical analysis but fails to improve Zero-Day generalization. Naive CoT lags behind standard methods, while RAG shows promise. Performance drops on Zero-Day data—suggesting possible leakage in older datasets—and on diverse sets like PrimeVul-Full, highlighting current struggles with varied vulnerability types.

Now let us look at LLMs' performance with **VSP strategies**. As shown in Table 3, VSP improves the F1 scores for some of the LLMs in certain evaluated settings. However, the extent of this improvement is variable; while gains are observed on existing datasets, they are often marginal on the unbiased Zero-Day dataset for some models. As Figure 6 shows, VSP helps address some limitations of existing prompting strategies and detects vulnerabilities more effectively in certain cases. For example, with VSP, GPT-3.5 focuses on vulnerability semantics and notices that the write of `&s->cmdbuf[s->cmdlen]` is unchecked and may have a buffer overflow issue. It pushes their F1 up to 62.68% (on GPT-4), 61.54% (on Claude3), 48.70% (on GPT-4), and 54.68% (on GPT-4)—from 54.55% (on

GPT-4), 48.64% (on GPT-4), 47.06% (on Llama2-13b), and 50.75% (on GPT-4)—for the four datasets, respectively. However, the improvement on the Zero-Day dataset, the dataset least likely to suffer from contamination, is relatively modest (1.64 percentage points, or 3.48% relative increase), indicating that VSP may not provide a significant benefit for vulnerability detection. Among individual LLMs, VSP achieves performance improvement by up to 25.62 points (or 111.6%) in F1 on Llama2-7b against the BigVul+CVEFixes dataset, compared to its best performance with existing strategies. On Claude3, the improvements are 27.05 and 28.37 points (or 114.5% and 85.52%) in F1 for the VulChecker and BigVul+CVEFixes datasets.

VSP also yields improvements in predictive reliability as measured by MCC and AuPRC for certain models. For example, on the Zero-Day dataset, VSP boosts Claude3's MCC from 0.0782 (Standard Few-Shot) to 0.2198, and improves its AuPRC to 0.6812, reinforcing that VSP helps models distinguish the positive class more effectively. On the challenging PrimeVul-Full dataset, VSP enables Gemini to achieve the highest MCC of 0.4237 and AuPRC of 0.4786, demonstrating that the vulnerability semantics focused guidance provided by VSP translates into more robust and correlated predictions across diverse vulnerability types.

<pre> 1 static xmlChar * 2 xmlParseAttValueComplex(xmlParserCtxtPtr ctxt, int *attlen, int normalize){ 3 xmlChar limit = 0; 4 xmlChar *buf = NULL; 5 size_t len = 0; 6 size_t buf_size = 0; 7 int c, l, in_space = 0; 8 28 buf_size = XML_PARSER_BUFFER_SIZE; 29 buf = (xmlChar *) xmlMallocAtomic(buf_size); 30 186 if ((in_space) && (normalize)) { 187 while (buf[len - 1] == 0x20) len--; 188 } 190 buf[len] = 0; </pre>
Answer with VSP: A: Step 1: Find vulnerability semantics: The function has input "len" and "buf". "len" is used to access the "buf" at line 187. Therefore, the vulnerability semantics is: <pre> 187 while (buf[len - 1] == 0x20) len--; </pre> Step 2: Vulnerability semantics analysis: At line 187, "buf" is accessed with "len". However, it is possible that "len" is less than or equal to 0, causing a out-of-bound read (CWE-125) vulnerability. Step 3: Patching strategy: To avoid the out-of-bound read (CWE-125) vulnerability at line 187, we should ensure that the "len" is greater than 0, which can be done by adding a condition at line 187. Therefore, the patch should be: Patch: <pre> - while (buf[len - 1] == 0x20) len--; + while (len > 0 && buf[len - 1] == 0x20) len--; </pre>
Answer with Standard Prompt: Patch: <pre> - if ((in_space) && (normalize)) { - while (buf[len - 1] == 0x20) len--; - } + if ((in_space) && (normalize) && len > 0) { + while (buf[len - 1] == 0x20 && len > 0) len--; + } </pre>
Answer with Naive CoT Prompt: Step 1: Analyze the code line by line: 1. This line declares a static function named "xmlParseAttValueComplex" that takes in three parameters: a pointer to a structure of type "xmlParserCtxtPtr" named "ctxt", a pointer to an integer "attlen", and an integer "normalize". The function returns a pointer to an "xmlChar". 2. The function's body begins on this line. 3. This line declares an "xmlChar" variable named "limit" and initializes it to 0. Step 2: Vulnerability semantics analysis: At line 187, the "buf" array is accessed at index "len - 1". However, it is possible that "len" is 0, causing an out-of-bound read (CWE-125) vulnerability. Step 3: Patching strategy: To avoid the out-of-bound read (CWE-125) vulnerability at line 187, we should add a condition to check if "len" is greater than 0 before accessing the "buf" array. If "len" is 0, we can skip the loop and proceed to the next line. Therefore, the patch should be: Patch: <pre> - while (buf[len - 1] == 0x20) len--; + if (len > 0) { + while (buf[len - 1] == 0x20) len--; + } </pre>

Fig. 8. GPT-3.5 fixed a CWE-125 vulnerability with VSP but not with Standard Prompting or naive CoT.

approach on unseen vulnerabilities. On the PrimeVul-Full dataset, VSP demonstrates strong generalization capabilities. Despite the increased difficulty of detecting 62 different CWEs, VSP still raise performance in some cases. For instance, GPT-4 improves from 28.26% (RAG) to 35.49% (VSP). Notably, Gemini achieves the highest

Overall, VSP improves less on Gemini (up to 7.24 points against the PrimeVul-Subset dataset) and GPTs (up to 16.64% against BigVul+CVEFixes for GPT-3.5) than the other LLMs. In comparison, zero-shot VSP (i.e., VSP without exemplars) does not help as much. It underperforms the best existing strategies on most of the LLM (e.g., GPT-4 achieves 9.05%-37.50% F1 with zero-shot VSP, while it achieves 46.07%-54.55% F1 with RAG and Naive CoT, against the four datasets), even often worse than standard prompting (e.g., Llama2-13b achieves 10.24%-25.71% F1 with zero-shot VSP, while with standard prompting, it achieves 14.11%-47.06% F1 against the four datasets). This indicates that the exemplars in VSP are essential for improving the LLMs for vulnerability detection.

Furthermore, when applying VSP to the fine-tuned model, we also observe performance gains, achieving F1 scores of 60.38%, 43.67%, 48.37%, and 46.43% across the four datasets. Notably, the base Llama2-13b model with VSP achieves an F1 score of 48.48% on the Zero-Day dataset, which is comparable to, and slightly higher than, the fine-tuned model with either standard prompting (46.30%) or VSP (48.37%). This finding is pivotal: it demonstrates that our proposed VSP strategy applied to an off-the-shelf open-source model can achieve performance parity with, or even surpass, a more resource-intensive fine-tuning

performance on this diverse dataset with an F1 of 41.72% using VSP. This suggests that the vulnerability semantics reasoning approach used in VSP is not limited to specific memory-related errors but helps LLMs reason more effectively about a wide variety of security flaws.

To validate these improvements statistically, we employed Wilcoxon signed-rank tests [89] to compute the p -value at the 0.95 confidence level, considering differences with $p \leq 0.05$ as significant. We used Cliff's Delta [19] to measure effect size, interpreting the strength as negligible if $|d| \leq 0.147$, small if $0.147 < |d| \leq 0.33$, medium if $0.33 < |d| \leq 0.474$, and large if $|d| > 0.474$ [78]. As shown in Table 4, the performance differences observed with VSP are statistically confirmed ($p < 0.001$) against existing prompting strategies. Specifically, in the aggregate analysis, VSP demonstrates a large effect size when compared to Standard Prompting ($d = -0.6413$ for F1) and Naive CoT ($d = -0.6066$ for F1). Even against RAG, VSP maintains a large effect size on F1 scores ($d = -0.5722$) and a medium effect size on MCC and AuPRC ($d = -0.4486$ and $d = -0.4555$), confirming that the vulnerability-semantics guidance offers a statistical advantage over general-purpose prompting strategies for detection.

Finding 2 (RQ2): VSP improves F1 scores over existing strategies, allowing resource-efficient off-the-shelf models to match or exceed fine-tuned performance on Zero-Day vulnerabilities. While vulnerability semantics focused guidance boosts detection across diverse types (PrimeVul-Full), improvements on the unbiased Zero-Day dataset are modest. Furthermore, VSP relies on exemplars for success, as the zero-shot version consistently underperforms.

3.2 Task 2: Vulnerability Classification

```
1 void ice_debugfs_module_write(struct file *filp,
2   const char __user *buf, size_t count, loff_t *ppos){
3   struct ice_pf *pf = file_inode(filp)->i_private;
4   struct dentry *dentry = file_dentry(filp);
5   struct device *dev = ice_pf_to_dev(pf);
6   char user_val[16], *cmd_buf;
7   int module, log_level, cnt;
8   ..... // array user_val is safely used without
9   overflowing the size 16.
```

zeroday_0229_detect/CWE125__CVE-2024-36935__linux
_vul.c:7: [2] (buffer) char: Statically-sized arrays can be improperly restricted, leading to potential overflows or other issues (CWE-119/CWE-120). Perform bounds checking, use functions that limit length, or ensure that the size is larger than the maximum possible length.

Fig. 9. An example where traditional static analyzer FlawFinder does not conduct comprehensive checks on the whole program but directly reports the potential vulnerability based on its predefined patterns, causing the false positive.

correct CWEs higher across different thresholds. Table 5 summarizes these metrics across all the CWEs. As micro F1 focuses on the global performance of the models, we mainly discuss micro F1 in the table.

With **existing prompting strategies**, the LLMs again see largely varying performance, from 4.39% (Llama-7b against VulChecker) to 41.11% (GPT-4 against VulChecker) in micro F1, across models of different vendors. GPT-4 again achieves the best performance against the VulChecker and BigVul+CVEFixes, with 41.11% and 34.44% micro F1. Among the four strategies, RAG performs the best on the VulChecker dataset, where four (Gemini, Claude3, Llama2-7b, Falcon) of the ten LLMs achieve the best micro F1 with RAG (28.74%, 31.97%, 19.51%, and 20.00% respectively), indicating that RAG mitigated insufficient context and improved the quality of few-shot learning. On the other datasets, naive CoT and standard prompting perform better in some cases. For example,

To evaluate the effectiveness of LLMs for vulnerability classification, we first measure the recall, precision, and F1 on each class (CWE) and then compute the overall effectiveness via macro and micro-averaging [8] as commonly done when evaluating multi-class classification tasks. Macro F1-score calculates the F1-score for each class independently and then takes the unweighted average, treating all classes equally. Micro F1-score, conversely, calculates global precision and recall by summing up the true positives, false positives, and false negatives across all classes, and then computes the F1-score from these global counts, effectively giving more weight to larger classes [8]. In addition to F1 scores, we report the Matthews Correlation Coefficient (MCC) and both Macro and Micro Area Under the Precision-Recall Curve (AuPRC). MCC offers a robust measure of the quality of multiclass classifications, while AuPRC evaluates the models' ability to rank

We also notice the challenge of classifying vulnerabilities across a wide range of types on the PrimeVul-Full dataset. With 62 different CWEs to distinguish between, most models struggle significantly compared to the 5-CWE classification task in other datasets. For instance, GPT-4's Micro F1 drops from 41.92% on PrimeVul-Subset (using Naive CoT) to just 18.76% on PrimeVul-Full. This sharp decline underscores that while LLMs can differentiate between a small set of distinct vulnerability types, their ability to accurately classify a broad spectrum of security flaws remains limited without specialized guidance.

Besides, we have similar observations to those on Task 1. For instance, between models from the same vendors, larger/newer ones again perform better (e.g., Llama2-13b achieved 19.73%-31.48% micro F1 while Llama2-7b only achieves 17.72%-28.37% micro F1). Yet generally the performance is not correlated consistently with model size, age, and capacity across vendors (e.g., GPT-3.5 achieved up to 36.26% micro F1 against the VulChecker dataset, while Claude3, the newer LLM achieved only up to 34.78% micro F1). Code-specific LLMs (CodeLlama, Qwen2.5-Coder, and DeepSeek-Coder-V2) generally exhibit lower micro F1 scores compared to the leading general-purpose commercial LLMs like GPT-4 and Claude3 (e.g., DeepSeek-Coder-V2 achieves up to 25.44% micro F1 while GPT-4 achieves 34.44% against BigVul+CVEFixes).

We also evaluate the capabilities of the fine-tuned Llama2-13b model. Overall, fine-tuning yields a substantial performance boost over the base Llama2-13b model across all datasets. For instance, on the Zero-Day dataset, the micro F1 score with standard prompting increases from 19.73% to 31.71%. When comparing different prompting strategies, we observe that the fine-tuned model is remarkably robust; the performance differences between Standard Prompting, Standard Few-Shot, Naive CoT, and RAG are minimal (e.g., ranging only from 31.41% to 31.90% on the Zero-Day dataset, and 32.47% to 33.72% on BigVul+CVEFixes). This indicates that the model has effectively internalized the classification task definition during training, reducing its reliance on in-context exemplars or retrieval mechanisms found in strategies like Few-Shot or RAG.

The MCC and AuPRC results for existing prompting strategies reveal significant instability in classification reliability, often more severe than F1 scores suggest. On the Zero-Day dataset, MCC values are frequently close to zero, indicating near-random classification capabilities for many models. For instance, Llama2-7b achieves a negligible MCC of 0.0116 with Standard Prompting, while even the sophisticated GPT-4 only reaches 0.1849. Furthermore, on the PrimeVul-Subset dataset, Llama2-7b even exhibits a negative MCC (-0.0058), implying a negative correlation between predictions and ground truth. The Micro AuPRC scores mirror this struggle; on the PrimeVul-Full dataset, which requires distinguishing among 62 classes, most models collapse, with Claude3 achieving a Micro AuPRC of just 0.0706 and Falcon reaching 0.1065. This highlights that without careful task-specific guidance, LLMs struggle to form distinct decision boundaries between vulnerability types.

Finding 3 (RQ1): For classification, with existing prompting strategies, the LLMs achieve up to 41.11% micro F1 (on GPT-4). Fine-tuning yields improvements for Llama2-13b and makes it robust to variations in prompting strategies. Data contamination in the non-zero-day datasets still exists, indicating the importance of using zero-day data for evaluation. Yet neither a consistently winning strategy, nor a consistent correlation between LLMs' size/age/max #tokens and their performance, is observed. The significant performance drop on PrimeVul-Full highlights the difficulty of broad-spectrum vulnerability classification.

With **VSP strategies**, the results in Table 5 reveal improvements brought by VSP for some of the LLMs and datasets studied. For example, with VSP, the classification performance is pushed to 47.27% micro F1 with Gemini, against BigVul+CVEFixes, where the best number from existing prompting strategies is 34.44% with GPT-4 (Standard Prompting). Against the Zero-Day dataset, the classification performance is pushed from 25.40% (Claude3 with standard prompting) to 40.16% (Claude3), indicating that VSP helps improve the effectiveness of vulnerability classification. As Figure 7 shows, Gemini correctly classifies a CWE-190 vulnerability with VSP while it fails with naive CoT: without focusing on vulnerability semantics, the LLMs may be easily distracted by the long source code contents and ignore the core part for vulnerability classification. However, Llama2-7b still

fails on this case. This indicates that the model’s capability still matters (in terms of size, as shown in Table 1): the model cannot reason about the vulnerability semantics correctly, causing the failure.

On the PrimeVul-Full dataset, absolute scores of VSP are lower due to the 62-class problem, VSP however achieves better effectiveness in some cases. For example, Gemini achieves a Micro F1 of 27.14% with VSP, compared to 10.56% with Standard Few-Shot. GPT-4 improves from 18.76% (Naive CoT) to 20.95% (VSP). This indicates that the structured reasoning provided by VSP helps models navigate the larger decision space of diverse CWEs more effectively than unguided prompting.

Furthermore, while the fine-tuned model is generally stable across existing strategies, VSP still manages to push the performance boundary further. On the BigVul+CVEFixes dataset, the fine-tuned model with VSP achieves a micro F1 of 35.53%, surpassing all other strategies. Similarly, on the Zero-Day dataset, VSP achieves the highest score of 32.02%. This demonstrates that even for a model well-tuned to the task, guiding the reasoning process via vulnerability semantics provides a tangible benefit over standard fine-tuning approaches.

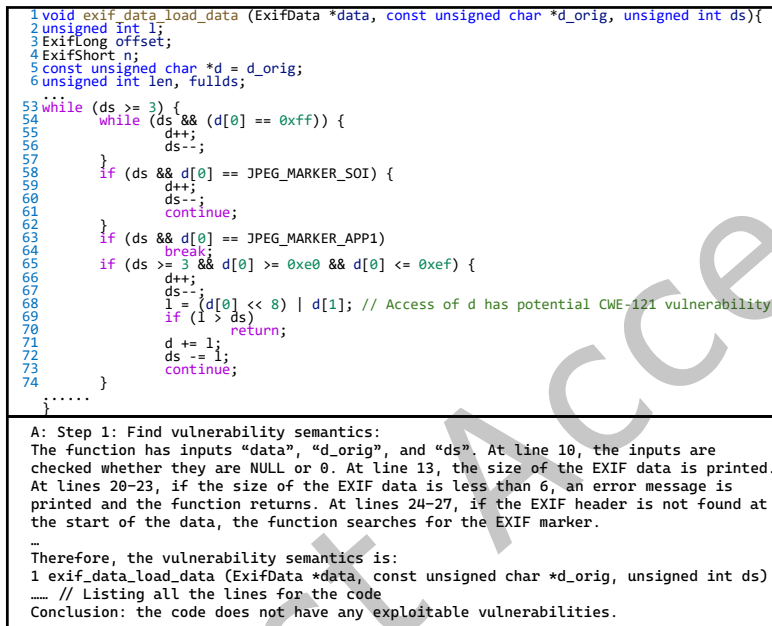


Fig. 10. GPT-3.5 with VSP outputs vague interpretation of code semantics on a vulnerability detection sample.

In 30 out of 40 cases, zero-shot VSP did not improve the LLM over the existing strategies, indicating that the exemplars in VSP were essential for improving vulnerability classification.

These observations are further supported by the statistical analysis in Table 4, which indicates that VSP achieves statistically distinguishable results ($p < 0.001$) compared to existing prompting strategies for classification. The effect sizes are predominantly medium to large; for instance, compared to Standard Prompting, VSP shows a medium effect size on Micro F1 ($d = -0.4436$) and Micro AuPRC ($d = -0.4387$). Against RAG, VSP exhibits a large effect size on both MCC ($d = -0.5131$) and Micro AuPRC ($d = -0.4952$). This suggests that for the multi-class classification task, the structured reasoning in VSP provides a robust advantage over retrieval-based or simple few-shot approaches.

VSP demonstrates a marked ability to improve the correlation and ranking quality of classification predictions as evidenced by MCC and AuPRC. On the Zero-Day dataset, VSP improves Claude3’s MCC from 0.0887 (Naive CoT) to 0.3345, and its Micro AuPRC from 0.4349 to 0.6160. This indicates that VSP not only helps the model select the correct CWE more often but also improves the calibration of its confidence scores. Furthermore, on the more challenging PrimeVul-Full dataset, VSP raises the Micro AuPRC for Gemini to 0.4544 (compared to 0.4455 with Standard Prompting) and GPT-4 to 0.4587, suggesting that the vulnerability semantics focused guidance provided by VSP helps maintain clearer decision boundaries even when the classification space is larger and more complex.

3.3 Task 3: Vulnerability Repair

For the ExtractFix dataset, we apply the LLM-generated patches to the vulnerable functions and use the test cases to validate if the vulnerabilities are removed and the programs are still functional. If the program passes the test cases after applying a patch, we consider the patch *valid*. However, a valid patch may not be correct [75] because it might alter or break the program functionality that is not covered by the test cases, despite passing them.

Therefore, we follow [32, 75] and manually inspect the valid patches, classifying them into one of four categories: (1) if the generated patch exactly matches the ground truth, it is considered syntactic equivalent (*SynEq*); (2) if not, but the behavior is the same as the ground truth, we consider it semantically equivalent (*SemEq*); (3) if the patch has different behavior from ground truth but still fixes the vulnerability without breaking code functionalities, we consider it plausible (*Plaus*); (4) if the patch can pass the test cases (i.e., the patch is valid) but breaks functionalities, we consider it unreasonable (*Unres*). Since the samples in BigVul+CVEFixes, Zero-Day and PrimeVul are not compilable and the test cases are not available, we directly inspect the generated patches and classify them into one of those categories. The process is first done by the first author and then confirmed by the second and third authors, with disagreements resolved via discussions. Only the *SynEq*, *SemEq*, and plausible patches are counted as *correct*, and we measure patching effectiveness in terms of *correct rate* = #correct patches/#testing samples. Falcon failed in all the settings on this task. Thus, we omit it in Table 6.

As Table 6 shows, with **existing prompting strategies**, the two Llama2 models barely work, only generating up to <10% (0.97–8.33%) correct patches against the four datasets. Using the existing strategies, patches from the four commercial models (Gemini, Claude3, and both GPTs) achieve up to 29.17–70.83% correct rates on the ExtractFix dataset, although much lower (10.68–33.96%) on the other datasets. Against the BigVul+CVEFixes, GPT-4 outperforms Claude3 (30.24% versus 20.76%). The largest LLM Gemini (with 50T parameters, as shown in Table 1) achieves a much lower best correct rate (10.68–29.17%) than the other three commercial LLMs (16.98–54.17% on Claude3, 18.45–50.00% on GPT-3.5, and 20.40–70.83% on GPT-4). Both GPT and Llama2 have their newer/larger versions performing better than the smaller ones. The code-specific LLMs (CodeLlama, Qwen2.5-Coder, and DeepSeek-Coder-V2) generally exhibit significantly lower correct rates compared to the leading general-purpose commercial LLMs, with their best performance being 12.50% on ExtractFix (Qwen2.5-Coder), and much lower on other datasets (e.g., 1.94–5.66% on BigVul+CVEFixes, Zero-Day, and PrimeVul-Subset). Via other metrics, it appears that all the LLMs can generate semantically equivalent and plausible patches besides the syntactic equivalent ones. The PrimeVul-Full dataset, with its 62 diverse CWEs, presents a significant challenge for vulnerability repair. Most models struggle to generate correct patches for this broad range of vulnerabilities. For example, GPT-4’s correct rate drops from 33.96% on PrimeVul -Subset (using RAG) to just 20.68% (using Standard Prompting) on PrimeVul-Full. This indicates that while LLMs can repair common vulnerabilities like buffer overflows, their ability to fix a wide variety of less common security flaws is limited. However, GPT-3.5 shows relative resilience, achieving a correct rate of 20.04% with standard prompting on this diverse dataset.

We also evaluate the performance of the fine-tuned Llama2-13b model with existing prompting strategies. With standard prompting, the fine-tuned model demonstrates a clear improvement over the base Llama2-13b model across all datasets. For example, on the BigVul+CVEFixes dataset, the correct rate increases from 1.27% (base) to 26.69% (fine-tuned). Similarly, on PrimeVul-Subset, the correct rate jumps from 3.77% to 15.09%. This substantial gain indicates that fine-tuning is highly effective for the complex task of vulnerability repair, enabling the model to learn the specific patterns and syntactic requirements of generating valid patches. Furthermore, the fine-tuned model shows a high degree of stability across different existing strategies (Standard Prompting, Standard Few-Shot, Naive CoT, and RAG), with correct rates often remaining within a close range (e.g., 25.85% to 27.12% on BigVul+CVEFixes).

Among the five strategies, standard prompting has the worst performance on the two Llama2s and Gemini (mostly near 0% correct rate), while zero-shot-completion performs the worst on other LLMs (0.00%–20.83%).

While patches generated by zero-shot-completion (s.2 template [75]) pass test cases in many ExtractFix samples, most of them are unreasonable (as marked in yellow). The reason is that, by removing the vulnerable code and letting the LLMs generate the fixed code, LLMs may generate code that alters the program's functionalities, although such code may still pass the test cases if the functionality tests are not comprehensive. Standard few-shot, Naive CoT, and RAG alternatively achieve the best correct rate. For example, Gemini achieves its best correct rates 10.68%-29.17% with Naive CoT. Claude3 achieves its best correct rates 54.17% and 20.76% with standard few-shot against the ExtractFix and BigVul+CVEFixes datasets, respectively, while it achieves the best correct rate 19.43% and 16.98% with RAG against the Zero-Day and PrimeVul-Subset dataset, respectively.

Between the datasets, the LLMs with any prompting strategy almost consistently perform better on the ExtractFix dataset (up to 70.83% correct rate) than on BigVul+CVEFixes (up to 30.24% correct rate). This is mainly because the ExtractFix dataset is a relatively simple testing set for traditional automatic vulnerability patching tools, while BigVul+CVEFixes is more complex which contains more challenging samples. Against the Zero-Day dataset, we notice performance drops compared to the BigVul+CVEFixes dataset in some cases (e.g., Gemini's 12.71% to 10.68% and Claude3's 20.76%-19.43%), indicating the potential data leakage/contamination issue. PrimeVul-Subset dataset also presents varied performance, with correct rates ranging from 3.77% to 33.96%.

Finding 5 (RQ1): For vulnerability repair, with existing prompting strategies GPT-4 achieves up to 70.83% correct rate. Fine-tuning boosts the performance of Llama2-13b and makes them robust to different prompting strategies. Standard few-shot, Naive CoT, and RAG alternatively achieve the best correct rate among different LLMs and datasets. Zero-shot completion is the worst on most LLMs, as many of its generated patches break the program and thus are unusable. The low correct rates on PrimeVul-Full highlight the difficulty of repairing diverse types of vulnerabilities.

With **VSP strategies**, some of the LLMs show improved correct rates against the four datasets (up to 83.33% on GPT-4. Figure 8 shows an example where VSP helps an LLM repair a vulnerability successfully while the existing prompting strategies do not. Relative to the best performance with existing strategies, VSP's improvements are up to 29.17 points (i.e., by 58.34%) in correct rate on GPT-3.5 against ExtractFix, and 12.50 points (by 17.65%) on GPT-4 against ExtractFix. On the two Llama2 models, the improvements are 100% (e.g., from 8.33% to 16.67%). The improvements on the Zero-Day dataset is large, from up to 20.40% (GPT-4 with RAG) to 31.07% (Claude3), indicating that VSP is effective for improvement zero-day vulnerability repair. Figure 8 shows an example for vulnerability repair. With VSP, GPT-3.5 successfully identifies the root cause of the buffer overflow and fixes it while Standard Prompting and Naive CoT simply add a boundary check without analyzing the vulnerability root cause correctly. Therefore, vulnerability-specific prompting adaptation is crucial for effective analysis. Furthermore, when applying VSP to the fine-tuned model, we observe additional performance gains. On the BigVul+CVEFixes dataset, the fine-tuned model with VSP achieves a correct rate of 30.08%, surpassing its best performance with existing strategies (27.12% with RAG). This again indicates that guiding the model's reasoning process with vulnerability semantics remains beneficial and complementary, even after the model has been explicitly fine-tuned for the repair task. On the PrimeVul-Full dataset, VSP again demonstrates its effectiveness. Despite the increased difficulty, VSP improves repair performance in many cases. For instance, GPT-3.5 achieves a correct rate of 29.00% with VSP, compared to 20.04% with Standard Prompting. GPT-4 improves from 20.68% (Standard Prompting) to 31.26% (VSP). This suggests that the structured reasoning provided by VSP helps models generate more accurate patches even for a diverse set of vulnerabilities. VSP also leads to increases in other metrics, especially the percentage of patches that are valid and syntactically/semantically equivalent (to ground truth). Zero-shot VSP, however, is not even as helpful as naive CoT in most cases, and almost always underperforms the best-performing existing prompting strategies.

Statistically, Table 4 shows that VSP yields distinguishable gains ($p < 0.001$) in the rate of correct patches compared to other strategies. The magnitude of this difference varies by baseline: VSP achieves a large effect size

0.3135 at temperature 1, with a corresponding drop in AuPRC from 0.5395 to 0.4810. Similarly, GPT-4 sees its MCC drop from 0.5903 to 0.5577 and its AuPRC from 0.6471 to 0.6152 on the same dataset as diversity increases. While the fine-tuned Llama2-13b model remains relatively stable—maintaining an MCC range of 0.5631–0.5787 on the VulChecker dataset—the general trend across commercial models suggests that higher entropy introduces noise that impairs the reliability of vulnerability detection.

Table 8 summarizes the influence of temperature on vulnerability classification effectiveness. Consistent with the detection task, we find that a lower temperature (specifically 0) typically yields the most robust results across the board. For instance, GPT-3.5 exhibits a decline in Micro F1 on the VulChecker dataset from 37.42% at temperature 0 to 35.44% at temperature 1. Similarly, on the Zero-Day dataset, Claude3 achieves Micro F1 of 40.16% at the default setting, while dropping to 38.72% as diversity increases. While GPT-4 shows a slight performance increase at higher temperatures on certain datasets (e.g., reaching 39.82% on Zero-Day at temperature 1 compared to 39.04% at 0), this behavior is not universal. Crucially, the fine-tuned Llama2-13b model and code-specific models like DeepSeek-Coder-V2 achieve their best Zero-Day performance (32.02% and 27.93%, respectively) at temperature 0. Consequently, a temperature of 0 remains the optimal choice to balance effectiveness with the reproducibility required for rigorous evaluation.

The impact of temperature on classification reliability is further illuminated by the MCC and AuPRC metrics. Higher temperatures tend to destabilize the correlation between predicted and actual vulnerability types. On the Zero-Day dataset, Claude3 shows a decline in MCC from 0.3345 (at temperature 0) to 0.2751 (at temperature 1), with Micro AuPRC falling from 0.6160 to 0.5713. While some models like the fine-tuned Llama2-13b show resilience—or even improvement in on BigVul+CVEFixes at higher temperatures—the overarching trend supports the use of deterministic settings to maintain consistent and precise classification boundaries.

Table 9 illustrates the effect of temperature on vulnerability repair performance. Consistent with the findings for detection and classification, the deterministic setting (temperature 0) tends to be the most effective for the majority of models, particularly the high-performing commercial ones. For instance, Claude3 sees a reduction in correct patches on the Zero-Day dataset from 31.07% at the default setting to 29.13% at temperature 1. While certain code-specific models like DeepSeek-Coder-V2 exhibit minor fluctuations—occasionally benefiting slightly from higher diversity (e.g., improving from 3.88% to 4.85% on Zero-Day)—these gains are inconsistent across datasets. The fine-tuned Llama2-13b model also favors the default temperature setting on the large-scale BigVul+CVEFixes dataset (30.08% at temperature 0 vs. 29.24% at temperature 1).

The statistical analysis results in Table 10 corroborate the stability of VSP across various temperature settings. For vulnerability detection, the performance differences between the default temperature (0) and higher settings are generally not statistically significant ($p > 0.05$), with the exception of the most diverse setting (0 vs 1). In classification, results are similarly stable, with most comparisons failing to reach statistical significance. While the differences in repair performance are statistically distinguishable ($p < 0.001$), the effect sizes across all three tasks are consistently negligible ($|d| \leq 0.015$).

Finding 7 (RQ3): Lower temperatures yield more robust and optimal effectiveness across vulnerability detection, classification, and repair. Increasing the temperature to introduce diversity generally degrades performance or offers inconsistent gains (although the differences/changes are mostly not statistically significant nor large), indicating that deterministic generation is preferable for vulnerability analysis to ensure both stable performance and reproducibility.

4.2 Impact of Top-P

We then examine the impact of Top-P. We again evaluate values of 0, 0.25, 0.5, 0.75, and 1.

Table 11 summarizes the performance of LLMs across different Top-P values for vulnerability detection. Unlike temperature, where the lowest setting was almost universally superior, the impact of Top-P is more nuanced. For

Table 16. Impact of system prompt (SP) on the LLM performance of vulnerability repair (with VSP). NA: No System Prompt; HA: Helpful Assistant; SE: Security Expert

LLM	SP	ExtractFix Dataset						BigVul-CVEFixes				Zero-Day				PrimeVul-Subset				PrimeVul-Full			
		Valid	SynEq	SemEq	Plaus	Unres	Correct	SynEq	SemEq	Plaus	Correct	SynEq	SemEq	Plaus	Correct	SynEq	SemEq	Plaus	Correct	SynEq	SemEq	Plaus	Correct
Gemini	NA	33.33%	12.50%	12.50%	8.33%	0.00%	33.33%	6.36%	5.51%	1.27%	13.14%	2.91%	4.85%	7.77%	15.53%	3.77%	3.77%	11.32%	18.87%	0.42%	3.94%	7.90%	12.26%
	HA	33.33%	12.50%	12.50%	8.33%	0.00%	33.33%	6.36%	5.93%	1.69%	13.98%	2.91%	4.85%	7.77%	15.53%	3.77%	5.66%	11.32%	20.75%	0.42%	3.94%	7.90%	12.26%
	SE	37.50%	12.50%	16.67%	8.33%	0.00%	37.50%	6.36%	5.93%	1.69%	13.98%	2.91%	4.85%	7.77%	15.53%	3.77%	5.66%	11.32%	20.75%	0.42%	3.94%	7.90%	12.26%
Claude3	NA	66.67%	25.00%	20.83%	20.83%	0.00%	66.67%	7.20%	9.32%	6.36%	22.88%	11.65%	9.71%	8.74%	30.10%	3.77%	5.66%	9.43%	18.87%	3.68%	3.68%	6.04%	13.40%
	HA	66.67%	25.00%	20.83%	20.83%	0.00%	66.67%	7.20%	9.75%	6.36%	23.31%	12.62%	9.71%	7.77%	30.10%	3.77%	5.66%	9.43%	18.87%	3.68%	3.45%	6.27%	13.40%
	SE	66.67%	25.00%	20.83%	20.83%	0.00%	66.67%	7.20%	9.75%	6.36%	23.31%	12.62%	9.71%	7.77%	30.10%	3.77%	5.66%	9.43%	18.87%	3.68%	3.23%	6.27%	13.17%
GPT-3.5	NA	79.17%	25.00%	33.33%	20.83%	0.00%	79.17%	9.75%	8.47%	12.29%	30.51%	14.56%	4.85%	23.30%	42.72%	7.55%	7.55%	16.98%	32.08%	0.22%	15.39%	13.38%	29.00%
	HA	79.17%	25.00%	33.33%	20.83%	0.00%	79.17%	9.75%	8.47%	12.29%	30.51%	14.56%	5.83%	23.30%	43.69%	7.55%	7.55%	16.98%	32.08%	0.22%	15.39%	13.38%	29.00%
	SE	79.17%	25.00%	33.33%	20.83%	0.00%	79.17%	9.75%	8.47%	12.29%	30.51%	15.53%	5.83%	24.27%	45.63%	7.55%	7.55%	16.98%	32.08%	0.00%	15.39%	13.38%	28.77%
GPT-4	NA	83.33%	25.00%	41.67%	16.67%	0.00%	83.33%	17.02%	12.77%	19.15%	48.93%	16.50%	6.80%	20.39%	43.69%	7.55%	9.43%	20.75%	37.74%	1.84%	7.36%	22.07%	31.26%
	HA	83.33%	25.00%	41.67%	16.67%	0.00%	83.33%	17.02%	12.77%	19.57%	49.15%	16.50%	6.80%	20.39%	43.69%	7.55%	9.43%	20.75%	37.74%	1.84%	7.36%	22.07%	31.26%
	SE	83.33%	25.00%	41.67%	16.67%	0.00%	83.33%	17.02%	12.77%	19.57%	49.15%	17.48%	5.83%	21.36%	44.66%	7.55%	9.43%	20.75%	37.74%	1.84%	7.36%	22.07%	31.26%
Llama2-7b	NA	16.67%	8.33%	4.17%	4.17%	0.00%	16.67%	2.12%	1.27%	0.00%	3.39%	0.00%	0.97%	1.94%	2.91%	1.89%	0.00%	5.66%	7.55%	1.84%	0.00%	5.06%	6.90%
	HA	16.67%	8.33%	4.17%	4.17%	0.00%	16.67%	2.54%	1.27%	0.00%	3.81%	0.00%	0.97%	1.94%	2.91%	1.89%	1.89%	5.66%	9.43%	1.84%	0.00%	5.06%	6.90%
	SE	20.83%	8.33%	4.17%	8.33%	0.00%	20.83%	2.54%	1.69%	0.00%	4.24%	0.97%	0.97%	1.94%	3.88%	1.89%	1.89%	5.66%	9.43%	1.84%	0.00%	4.84%	6.68%
Llama2-13b	NA	16.67%	12.50%	4.17%	0.00%	0.00%	16.67%	2.12%	1.27%	0.00%	3.81%	0.00%	0.97%	1.94%	2.91%	1.89%	1.89%	5.66%	11.32%	0.00%	1.27%	6.63%	7.90%
	HA	16.67%	12.50%	4.17%	0.00%	0.00%	16.67%	2.12%	1.27%	0.42%	3.81%	0.00%	0.97%	1.94%	2.91%	1.89%	3.77%	5.66%	11.32%	0.22%	1.27%	6.63%	8.12%
	SE	16.67%	12.50%	4.17%	0.00%	0.00%	16.67%	2.12%	1.27%	0.42%	3.81%	0.00%	0.97%	1.94%	2.91%	1.89%	3.77%	5.66%	11.32%	0.22%	1.27%	6.63%	8.12%
CodeLlama	NA	16.67%	8.33%	4.17%	4.17%	0.00%	16.67%	0.00%	0.85%	1.69%	2.54%	0.00%	0.97%	1.94%	2.91%	1.89%	1.89%	1.89%	5.66%	0.00%	1.69%	2.32%	4.01%
	HA	16.67%	8.33%	4.17%	4.17%	0.00%	16.67%	0.00%	0.85%	2.12%	2.97%	0.00%	1.94%	1.94%	3.88%	1.89%	1.89%	1.89%	5.66%	0.00%	1.46%	2.32%	3.78%
	SE	16.67%	8.33%	4.17%	4.17%	0.00%	16.67%	0.00%	0.85%	2.12%	2.97%	0.00%	1.94%	1.94%	3.88%	1.89%	1.89%	1.89%	5.66%	0.00%	1.46%	2.32%	3.78%
QC-2.5	NA	16.67%	4.17%	4.17%	8.33%	0.00%	16.67%	0.85%	1.27%	2.54%	4.66%	0.97%	1.94%	1.94%	4.85%	1.89%	3.77%	1.89%	7.55%	0.00%	2.21%	3.49%	5.70%
	HA	16.67%	4.17%	4.17%	8.33%	0.00%	16.67%	0.85%	1.27%	2.54%	4.66%	0.97%	0.97%	2.91%	4.85%	1.89%	3.77%	1.89%	7.55%	0.00%	2.21%	3.49%	5.70%
	SE	16.67%	4.17%	4.17%	8.33%	0.00%	16.67%	0.85%	1.27%	2.54%	4.66%	0.97%	0.97%	2.91%	4.85%	1.89%	3.77%	1.89%	7.55%	0.00%	2.21%	3.49%	5.70%
DSC-V2	NA	12.50%	4.17%	4.17%	4.17%	0.00%	12.50%	1.27%	0.85%	2.12%	4.24%	0.97%	0.97%	1.94%	3.88%	1.89%	1.89%	1.89%	5.66%	0.21%	1.85%	2.05%	4.10%
	HA	12.50%	4.17%	4.17%	4.17%	0.00%	12.50%	1.27%	0.85%	2.12%	4.24%	0.97%	0.97%	1.94%	3.88%	1.89%	1.89%	3.77%	7.55%	0.21%	1.85%	2.05%	4.10%
	SE	12.50%	4.17%	4.17%	4.17%	0.00%	12.50%	1.27%	0.85%	2.12%	4.24%	0.97%	1.94%	0.97%	3.88%	1.89%	0.00%	3.77%	5.66%	0.21%	1.85%	2.05%	4.10%
FT-Llama2	NA	25.00%	4.17%	12.50%	8.33%	0.00%	25.00%	0.00%	14.41%	15.68%	30.08%	0.00%	4.85%	13.59%	18.45%	0.00%	9.43%	18.87%	28.30%	0.63%	13.08%	11.38%	25.09%
	HA	16.67%	0.00%	12.50%	4.17%	0.00%	16.67%	0.00%	13.98%	15.68%	29.66%	0.00%	4.85%	13.59%	18.45%	0.00%	9.43%	20.75%	28.30%	0.63%	13.08%	11.38%	25.09%
	SE	20.83%	0.00%	16.67%	4.17%	0.00%	20.83%	0.00%	13.98%	15.25%	29.24%	0.00%	5.83%	13.59%	19.42%	0.00%	7.55%	22.64%	28.30%	0.63%	13.08%	11.38%	25.09%

nuanced picture. While commercial models like GPT-4 see a modest improvement in F1 scores with the 'Security Expert' persona, the MCC and AuPRC metrics indicate that this boost is not consistently translated into more robust predictions. For instance, GPT-4's MCC on the PrimeVul-Subset dataset increases from 0.5329 (No System Prompt) to 0.5468 with the 'Security Expert' persona, while AuPRC improves from 0.5546 to 0.5668.

<pre> 1 int isoent_gen_joliet_identifier(struct archive_write *a, struct isoent *isoent, struct idr *idr){ 2 struct iso9660 *iso9660; 3 struct isoent *np; 4 unsigned char *p; 5 size_t l; 6 int f; 7 int fmax, parent_len; 17 r = idr_start(a, idr, isoent->children.cnt, fmax, 6, 2, &rb_ops); 23 for (np = isoent->children.first; np != NULL; np = np->chnext){ 24 unsigned int ext_off, noff, weight; 77 if ((int)l == fmax) 78 noff = ext_off + 6; // Calculation of noff has potential CWE-190 vulnerability 79 else if ((int)l == fmax - 2) 80 noff = ext_off + 4; 81 else if ((int)l == fmax - 4) 82 noff = ext_off + 2; 83 else 84 noff = ext_off; 85 idr_register(idr, np, weight, noff); 86 } } </pre>	A: Step 1: Find vulnerability semantics: The function has inputs "a", "isoent", and "idr". At line 17, the function starts an "idr" with a count of "isoent->children.cnt", a maximum size of "fmax", and other parameters. Then, at lines 23-86, a loop iterates over the children of "isoent" and performs various operations on them. Therefore, the vulnerability semantics is: <pre> 1 int isoent_gen_joliet_identifier(struct archive_write *a, struct isoent *isoent, struct idr *idr) 17 r = idr_start(a, idr, isoent->children.cnt, fmax, 6, 2, &rb_ops); 23 for (np = isoent->children.first; np != NULL; np = np->chnext) 85 idr_register(idr, np, weight, noff); </pre> Step 2: Vulnerability semantics analysis: At line 17, the "idr" is started with a count of "isoent->children.cnt" and a maximum size of "fmax". However, the code does not check whether "isoent->children.cnt" is greater than "fmax", which may cause a heap-based buffer overflow (CWE-122) vulnerability. Conclusion: the code has a CWE-122 vulnerability.
---	---

Fig. 11. GPT-3.5 identifies an incorrect vulnerability cause on a vulnerability classification sample.

exactly how to perform the task during its training, adding extra instructions to ask the LLM to 'act like an expert' does not improve its performance.

Table 15 details the effect of system prompts on vulnerability classification. Similar to the detection task, the "Security Expert" persona provides benefits for specific models but lacks universal effectiveness. Claude3, for instance, exhibits a clear improvement on the Zero-Day dataset, rising from 40.16% Micro F1 without a system prompt to 41.46% with the "Security Expert" persona. Conversely, GPT-3.5 shows a slight decline on the same dataset (19.23% vs. 18.96%) when the persona is applied. The fine-tuned Llama2-13b model demonstrates remarkable stability, with its Zero-Day performance remaining virtually unchanged (32.02% vs. 31.96%). This indicates that because the fine-tuned model has already learned ex-

Table 16 presents the impact of system prompts on vulnerability repair. For this complex generative task, we observe that system prompts have a very limited and inconsistent effect across most models. High-performing commercial models like GPT-4 and Claude3 demonstrate remarkable stability, with their correct rates remaining almost entirely unchanged regardless of the persona assigned. This suggests that their repair capabilities are deeply ingrained and not easily influenced by high-level instructions. A notable exception is GPT-3.5, which shows a tangible improvement on the Zero-Day dataset when primed with the "Security Expert" persona, increasing its correct rate from 42.72% to 45.63%. This indicates that for some models, persona-based guidance can be beneficial when tackling novel repair scenarios. Conversely, for the fine-tuned Llama2-13b model, system prompts can be slightly detrimental, causing a minor performance regression on datasets like BigVul+CVEFixes (30.08% vs. 29.24%).

Table 10 indicates that while adding a persona can lead to statistically distinguishable results—particularly for vulnerability detection ($p < 0.001$)—the magnitude of this change is limited. The effect sizes for all pairwise comparisons (No Prompt vs. Helpful Assistant, No Prompt vs. Security Expert, and Helpful Assistant vs. Security Expert) are negligible across detection, classification, and repair tasks ($|d| < 0.05$). For classification specifically, the statistical significance varies by metric, further suggesting that system prompts do not fundamentally shift the performance baseline established by the VSP strategy.

Finding 9 (RQ3): System prompts offer modest gains for general-purpose models on novel data, but often show negligible impact or slight regression for code-specific and fine-tuned models.

5 RQ4: LLMs vs. Traditional Approaches

Tables 17, 18, and 19 compare LLMs to traditional approaches on the three tasks. The best key results are marked in boldface. VulChecker, ExtractFix, and VulnFix require input programs to be compilable whereas the BigVul+CVEFixes, Zero-Day, and PrimeVul samples are not. Thus, corresponding results are marked as none (-).

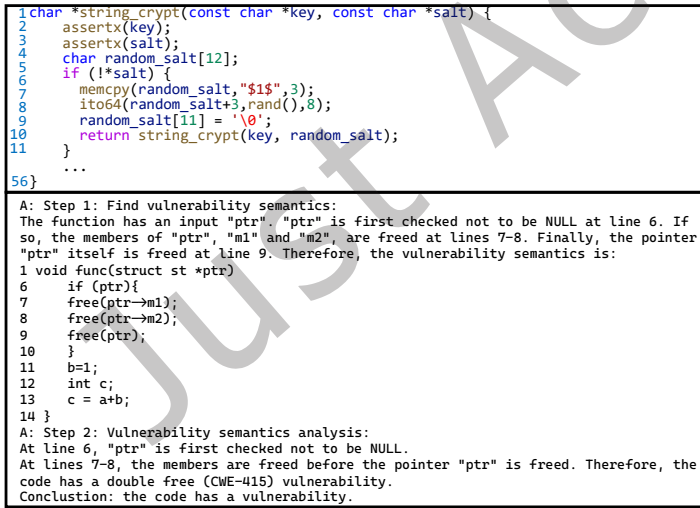


Fig. 12. Falcon generates unrelated outputs during vulnerability detection.

ability detectors, Devign and LineVul, achieve lower F1 compared to the one reported in their original papers. The reason is that their original testing sets have the overfitting issue, which inflates their effectiveness [68, 71].

With **existing prompting strategies**, the LLMs are close (e.g., 48.64% versus 48.92% on BigVul+ CVEFixes) or superior (e.g., 54.55% versus 52.31%, 47.06% versus 43.79%, 50.75% versus 13.04%, and 33.98% versus 9.56% on the VulChecker, Zero-Day, PrimeVul-Subset, and PrimeVul-Full datasets, respectively) to the best traditional approaches (CodeQL, FlawFinder, and Devign) in terms of F1 for vulnerability detection. This indicates the limitations of traditional static analyzers and the advantages of LLMs for vulnerability analysis. Figure 9 shows an example: FlawFinder does not conduct comprehensive checks on the whole program but directly reports the potential vulnerability based on its predefined patterns, causing the false positive. We notice that two of the DL-based vulner-

Table 17. Comparison to traditional approaches on vulnerability detection

Approach	VulChecker Dataset					BigVul+CVEFixes					Zero-Day					PrimeVul-Subset					PrimeVul-Full				
	Recall	Precision	F1	MCC	AuPRC	Recall	Precision	F1	MCC	AuPRC	Recall	Precision	F1	MCC	AuPRC	Recall	Precision	F1	MCC	AuPRC	Recall	Precision	F1	MCC	AuPRC
best LLM w/ existing	60.00%	50.00%	54.55%	0.4993	0.5677	40.91%	60.00%	48.64%	0.4792	0.5159	47.06%	47.06%	47.06%	-0.0588	0.6030	45.29%	57.72%	50.75%	0.4996	0.5231	60.69%	23.59%	33.98%	0.3526	0.4266
best LLM with VSP	70.00%	56.75%	62.68%	0.5903	0.6471	51.28%	76.92%	61.54%	0.6164	0.6504	44.36%	54.09%	48.70%	0.0822	0.6564	60.39%	49.95%	54.68%	0.5357	0.5576	63.75%	31.00%	41.72%	0.4237	0.4786
Devign (Pretrained)	26.80%	30.95%	28.72%	0.2240	0.3212	28.09%	83.33%	42.01%	0.4737	0.5709	49.33%	39.36%	43.79%	-0.2757	0.5701	3.22%	38.59%	5.95%	0.1053	0.2233	5.06%	21.48%	8.19%	0.0929	0.1453
Devign (Retrained)	28.94%	31.57%	30.20%	0.2375	0.3341	39.61%	64.16%	48.98%	0.4681	0.5456	51.80%	39.36%	44.73%	0.3898	0.4772	11.40%	25.47%	15.75%	0.1187	0.2236	10.98%	15.04%	12.69%	0.0571	0.1696
LineVul (Pretrained)	1.18%	50.00%	2.30%	0.0663	0.2997	5.65%	92.98%	10.65%	0.2243	0.5113	3.88%	50.00%	3.88%	0.0000	0.5097	26.39%	5.67%	9.34%	0.0703	0.1712	21.68%	5.73%	9.06%	0.0638	0.1475
LineVul (Retrained)	1.36%	58.00%	2.65%	0.0788	0.3405	7.85%	61.37%	13.92%	0.1978	0.3870	4.07%	57.50%	7.61%	0.1360	0.3504	26.92%	4.54%	7.76%	-0.1608	0.1897	25.80%	3.84%	6.68%	-0.2151	0.1811
VulChecker	95.29%	15.76%	27.04%	0.2606	0.5573	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
CppCheck	1.18%	50.00%	2.30%	0.0663	0.2997	4.05%	77.55%	7.70%	0.1719	0.4265	7.77%	50.00%	13.45%	0.0000	0.5194	2.78%	1.43%	1.89%	-0.0178	0.0354	1.74%	5.28%	2.62%	0.0153	0.0482
FlawFinder	40.00%	75.56%	52.31%	0.5207	0.6044	26.65%	98.04%	41.91%	0.5035	0.6376	21.36%	55.00%	30.77%	0.0491	0.5784	23.61%	6.85%	10.63%	0.0827	0.1636	3.91%	6.85%	4.98%	0.0323	0.0666
CodeQL	14.12%	85.71%	24.24%	0.3291	0.5372	33.69%	89.27%	48.92%	0.5391	0.6276	17.48%	47.37%	25.53%	-0.0250	0.5306	69.44%	7.19%	13.04%	0.1655	0.3877	49.54%	5.29%	9.56%	0.0941	0.2809

Table 18. Comparison to traditional approaches on vulnerability classification

Approach	VulChecker Dataset					BigVul+CVEFixes					Zero-Day					PrimeVul-Subset					PrimeVul-Full				
	Macro F1	Micro F1	MCC	MacAu	MicAu	Macro F1	Micro F1	MCC	MacAu	MicAu	Macro F1	Micro F1	MCC	MacAu	MicAu	Macro F1	Micro F1	MCC	MacAu	MicAu	Macro F1	Micro F1	MCC	MacAu	MicAu
best LLM w/ existing	38.28%	41.11%	0.3125	0.4957	0.5528	26.91%	34.44%	0.2213	0.4644	0.5579	19.57%	31.90%	0.2045	0.2858	0.5466	30.33%	38.92%	0.2570	0.3391	0.5699	5.03%	18.76%	0.1129	0.0670	0.4495
best LLM with VSP	44.46%	42.10%	0.3133	0.5147	0.5752	39.47%	47.27%	0.3544	0.4523	0.5903	37.16%	40.16%	0.3345	0.4677	0.6160	30.73%	39.53%	0.2625	0.4066	0.5875	5.59%	20.95%	0.1690	0.1399	0.4587
VulChecker	30.98%	36.23%	0.2474	0.4057	0.5131	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
CppCheck	0.00%	0.00%	0.0329	0.0350	0.0257	1.22%	1.89%	0.0282	0.0180	0.0698	1.48%	1.90%	0.0269	0.0185	0.0718	0.00%	0.00%	0.0118	0.0308	0.0052	0.00%	0.00%	0.0231	0.0008	0.0314
FlawFinder	8.15%	12.16%	0.0349	0.2416	0.3890	12.01%	16.40%	0.0452	0.2541	0.3281	19.16%	12.02%	0.0332	0.2299	0.3701	0.00%	0.00%	0.0206	0.0096	0.0180	0.00%	0.00%	0.0378	0.0088	0.0040
CodeQL	2.50%	2.29%	0.0321	0.0304	0.0805	0.49%	0.42%	0.0059	0.0070	0.0523	0.00%	0.00%	0.0201	0.0353	0.0312	0.00%	0.00%	0.0131	0.0208	0.0466	0.00%	0.00%	-0.0152	0.0343	0.0347

Table 19. Comparison to traditional approaches on vulnerability repair

Approach	ExtractFix Dataset						BigVul+CVEFixes						Zero-Day						PrimeVul-Subset						PrimeVul-Full					
	Valid	SynEq	SemEq	Plaus	Unres	Correct	SynEq	SemEq	Plaus	Correct	SynEq	SemEq	Plaus	Correct	SynEq	SemEq	Plaus	Correct	SynEq	SemEq	Plaus	Correct	SynEq	SemEq	Plaus	Correct				
best LLM w/ existing	54.17%	8.33%	37.50%	8.33%	0.00%	54.17%	9.30%	10.47%	13.95%	33.72%	19.42%	6.80%	14.56%	40.78%	0.00%	18.87%	15.09%	33.96%	0.00%	6.33%	14.35%	20.04%	0.00%	6.33%	14.35%	20.04%				
best LLM with VSP	83.33%	25.00%	41.67%	16.67%	0.00%	83.33%	17.02%	12.77%	19.15%	48.93%	16.50%	6.80%	20.39%	43.69%	0.00%	9.43%	28.30%	37.74%	1.84%	7.36%	22.07%	31.26%	0.00%	7.36%	22.07%	31.26%				
VulRepair (Pretrained)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	21.19%	4.24%	2.54%	27.96%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%				
VulRepair (Retrained)	25.00%	4.17%	4.17%	16.67%	0.00%	25.00%	4.66%	4.66%	8.05%	17.37%	0.97%	2.91%	7.77%	11.65%	2.78%	4.17%	13.89%	20.83%	5.29%	5.75%	6.21%	17.24%	0.00%	5.75%	6.21%	17.24%				
ExtractFix	83.33%	20.83%	33.33%	29.16%	0.00%	83.33%	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-				
VulnFix	45.83%	20.83%	12.50%	12.50%	0.00%	45.83%	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-				

Retraining the DL-based models on DiverseVul generally improves their performance compared to the pre-trained versions on our datasets. For example, Devign (Retrained) achieves an F1 score of 48.98% on BigVul+CVE-Fixes, up from 42.01% in its pre-trained state. For vulnerability classification, the merits of LLMs are 4.88–18.04 points or by 13.5%–110% greater than the best baseline (VulChecker and FlawFinder) against the VulChecker, BigVul+CVEFixes, and Zero-Day datasets in micro F1. For PrimeVul-Subset and PrimeVul-Full, all three traditional tools fail completely with 0% accuracy. However, for the repair task, the best baseline repair performance (83.33%, from the tool ExtractFix) is higher than LLMs (70.83%) on the ExtractFix datasets. VulRepair achieves relatively high correct rate against BigVul+CVEFixes, due to its DL model overfitting to the training data—it works better on the BigVul+CVEFixes dataset which overlaps with its training set [29]. When we retrain VulRepair on DiverseVul (removing the overlap), its correct rate drops to 17.37% on BigVul+CVEFixes, but it achieves higher correct rates on other datasets.

Finding 10 (RQ4): LLMs with the best existing prompting strategies perform similarly to or better than the best traditional approaches on vulnerability detection, classification, and repair.

With **VSP strategies**, however, some of the top-performing LLMs achieve higher F1 scores than the evaluated traditional approaches across the three tasks. For example, 62.68/61.54/48.70/54.68/41.72% vs. 52.31/48.92/43.79/13.04/9.06% for detection, 42.10/47.27/40.16/39.53/23.32% vs. 36.23/ 16.40/12.02/0.00/0.00% for classification, and 83.33/32.37/30.25/37.74/31.26% vs. 83.33/30.50/ 0.00/0.00/0.00% for repair. For detection and classification, on which best existing strategies already suffice to make LLMs edge over them, VSP pushes up the margins. VSP also enables certain LLMs (e.g., GPT-4) to match the performance of ExtractFix, the most effective repair baseline, even though this tool takes advantage of tests to facilitate repair, which LLMs do not use/need. VSP also helps LLMs get closer to the best traditional approach on any of the three tasks.

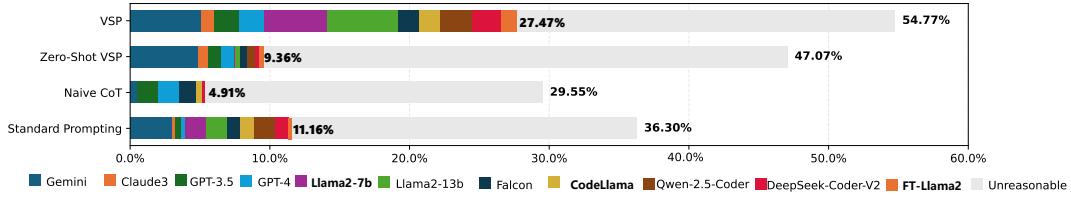


Fig. 13. Answer vs. reasoning correctness in vulnerability detection.

Finding 11 (RQ4): VSP enables certain LLMs to outperform or match the best traditional approaches across the evaluated tasks. For vulnerability detection, some VSP-enhanced LLMs (e.g., GPT-4) surpass the traditional approaches on the unbiased Zero-Day dataset, with larger margins observed on other datasets. For classification and repair, some VSP-enhanced LLMs also surpass the traditional approaches.

6 RQ5/RQ6: Correctness of Reasoning

In addition to correct (final) *answers*, correct *reasoning* also matters as it indicates better certainty hence gives users more trust of the answers [87]. We assess LLMs' reasoning correctness by examining whether their correct answers are reasonable (i.e., associated with correct reasoning). We then look at how VSP can improve LLMs in this regard too.

6.1 Quantitative Analysis of the Reasoning Correctness

We first conduct a quantitative analysis for the correctness of the reasoning. We manually examine the LLM-generated reasoning in each response. Given the manual nature and study scale, for each task and each LLM on each prompting strategy, we randomly sample the LLM responses for a sample size greater than the one statistically significant at 95% confidence level and 5% margin of error with respect to the population (i.e., total #responses by the LLM with the strategy)—466, 377, 247 for detection, classification, and repair, respectively.

We evaluate whether the generated reasoning captures the vulnerability semantics and justifies the outcome correctly. Since LLMs only generate final answers with standard few-shot and RAG (as the exemplars do), we only evaluate the other strategies. We notice that, on all the true-negative predictions, the reasoning is correct, and the reasoning for incorrect responses is consistently incorrect. Thus, we only show the results on the true-positive cases, as depicted in Figures 13, 14, and 15: the total bar length indicates the ratio of responses with correct answers (e.g., 29.55% on naive CoT in Fig. 13), while the solid-stacked-bar length indicates the ratio of responses with correct reasoning (e.g., 4.91%), for all the LLMs' respective responses combined on each strategy. Notably, for any LLM and strategy, *reasoning for incorrect answers is consistently incorrect*. We also note that the purpose of Figures 13, 14, and 15 is to illustrate the overall reasoning correctness for each prompting strategy, rather than to compare the reasoning correctness across different LLMs. We randomly sampled the correct prediction cases from all model outputs. The figures do not show the ratio of correct answers for each individual LLM.

With **existing prompting strategies**, the LLMs have up to 11.16%, 14.07%, and 7.57% of their responses backed by correct reasoning, although the ratios of correct answers are up to 36.30%, 25.02%, and 8.96%, for detecting, classifying, and repairing vulnerabilities, respectively. The ratio of correct reasoning for correct answers is $\frac{11.16\%}{36.30\%} = 30.74\%$ on the detection task; similarly, the ratio on classification and repair is 56.23% and 84.49%, respectively. That is, around 70% of the correct detection outcomes are not justified by correct reasoning, but less (<50%) for correct CWE prediction, and only a few (around 15%) for correct repair, which can be explained by the fact that the more challenging tasks require correct reasoning to provide correct answers. For instance, while it is possible to *guess* correctly in detection and classification tasks, a correct repair requires LLMs to

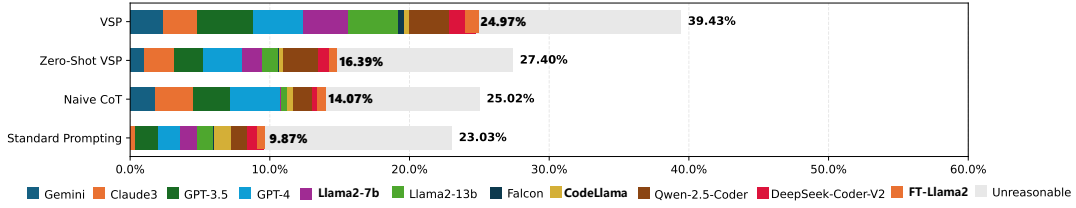


Fig. 14. Answer vs. reasoning correctness in vulnerability classification.

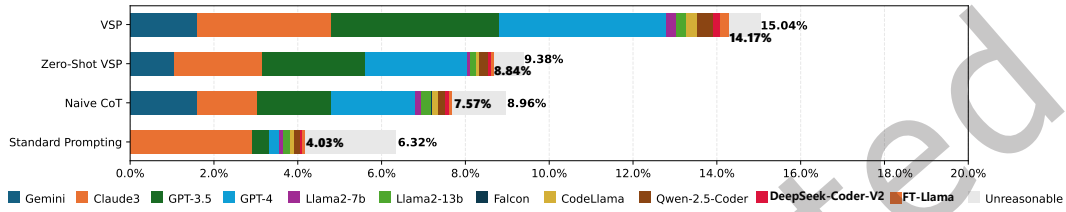


Fig. 15. Answer vs. reasoning correctness in vulnerability repair.

correctly analyze the code before generating the correct patches. However, there are still a few correct patches with incorrect reasoning; we found such cases happen when the patches are very simple and can be directly borrowed from seen training samples (e.g., change `<=` to `<`).

Our results also reveal that the LLMs have widely varying reasoning correctness for different tasks and with different prompting strategies. For instance, with standard prompting, Gemini has overall notably greater correctness than all the other LLMs for detection, where Claude3 performs the best in this regard for repair. We notice that Llama2 has much less correct reasoning. This indicates that it is necessary to have more powerful LLMs for vulnerability patching. Overall, code-specific LLMs (CodeLlama, Qwen-2.5-Coder, and DeepSeek-Coder-V2) have less correct reasoning compared to general LLMs (e.g., Gemini), especially on the task of vulnerability repair, indicating that code-specific LLMs are not good at code semantic understanding and data/control flow analysis. Gemini generates patches without providing reasoning with standard prompting for classification and repair, thus the respective numbers are not shown. Naive CoT improves over standard prompting for both classification and repair, but not for detection.

Finding 12 (RQ5): The evaluated LLMs frequently suffer incorrect reasoning, even for correct answers (71.1% has incorrect reasoning on vulnerability detection, 46.4% on classification, and 14.5% on repair).

With **VSP strategies**, the LLMs have their reasoning correctness improved, both individually and collectively, raising the correctness ratios to 27.47%, 24.97%, and 14.17% for detection, classification, and repair (from 11.16%, 14.07% and 7.57% with existing strategies). Behind the *correct* answers, the reasoning correctness ratios are improved by VSP to $\frac{27.47\%}{54.77\%} = 50.15\%$ for vulnerability detection, and 63.33% and 94.22% for classification and repair, respectively. Without using exemplars, zero-shot VSP also boosts the LLMs' reasoning for classification and repair (from 14.07% to 16.39% and 7.57% to 8.84%), but not for detection (9.36% versus 11.16%).

Finding 13 (RQ6): VSP improves the LLMs in correct reasoning, especially for correct answers, by up to 63.14%, 12.62%, and 11.51% for vulnerability detection, classification, and repair, respectively.

6.2 Qualitative Analysis of the Incorrect Reasoning

While our quantitative results show that VSP improves overall prediction accuracy and reasoning correctness for some of the LLMs, we further conduct deeper qualitative analysis revealing specific types of reasoning errors

that persist, even when VSP is applied. We manually examined a subset of LLM responses, focusing on cases where the reasoning was incorrect. As a result, we classify them into the following categories.

Vague Interpretation of Code Semantics. A prevalent type of incorrect reasoning stems from the LLMs' vague interpretation of code semantics, even when guided by VSP. This often makes the LLMs ignore the vulnerable code and directly output unreliable conclusions. For example, in Figure 10, the actual vulnerability is a CWE-121 at line 68. This occurs because the `d` pointer and `ds` length can be manipulated by the preceding while loops such that `d[0]` or `d[1]` accesses memory beyond the bounds of the `d_orig` buffer. However, GPT-3.5, despite using VSP, incorrectly concludes that "the code does not have any exploitable vulnerabilities." Its reasoning is vague, just going through the code like without actual vulnerability analysis, completely missing the critical data flow related to `d` and `ds` and their impact on the array access.

We observe that vague interpretation is not monolithic; rather, it manifests through several recurring reasoning patterns across different models and samples. In many cases, the LLM's reasoning remains at the level of functional description, accurately restating what the code does line by line, yet failing to transition from what happens to why it is security-relevant. Variables influenced by external inputs, memory accesses, or boundary-sensitive operations are described neutrally rather than being identified as potential sources of vulnerability.

A common manifestation of vagueness is the model's failure to prioritize security-critical elements within the code. Although multiple variables, loops, or memory operations may be present, the LLM often treats them as equally important, rather than identifying a small subset of security-relevant statements (e.g., buffer writes, pointer dereferences, or loop bounds derived from external input) that dominate the vulnerability semantics.

```
1 gchar *rsvg_io_get_file_path(const gchar *filename, const gchar
2 *base_uri){
3     gchar *absolute_filename;
4     if (g_file_test(filename, G_FILE_TEST_EXISTS) ||
5         g_path_is_absolute(filename))
6         absolute_filename = g_strdup(filename);
7     else{
8         gchar *tmpcdir;
9         gchar *base_filename;
10        if (base_uri){
11            base_filename = g_filename_from_uri(base_uri, NULL,
12 NULL);
13        }
14        if (base_filename != NULL){
15            tmpcdir = g_path_get_dirname(base_filename);
16            g_free(base_filename); // First free
17        }
18        else
19            return NULL;
20        }
21        else{
22            tmpcdir = g_get_current_dir();
23            absolute_filename = g_build_filename(tmpcdir, filename,
24 NULL);
25            g_free(base_filename); //Second free
26        }
27        return absolute_filename;
28 }
```

Answer by GPT-3.5:

A: Step 1: Find vulnerability semantics:
The function has inputs "filename" and "base_uri". If "filename" exists or is an absolute path, it is duplicated and assigned to "absolute_filename" at line 6. Otherwise, "tmpcdir" is assigned with the directory name of "base_filename" at line 14. Then, "absolute_filename" is built with "tmpcdir" and "filename" at line 22. Finally, "absolute_filename" is returned. Therefore, the vulnerability semantics is:
Step 2: Vulnerability semantics analysis:
At line 6, "filename" is duplicated and assigned to "absolute_filename".
At line 14, "tmpcdir" is assigned with the directory name of "base_filename".
At line 22, "absolute_filename" is built with "tmpcdir" and "filename".
There is no vulnerability in this code because there is only one free operation at line 24.
Conclusion: the code does not have an exploitable vulnerability.

In some instances, the reasoning appears to rely on superficial pattern recognition (e.g., detecting the presence of arrays or loops) without reconstructing the underlying data or control dependencies that make a specific operation unsafe. As a result, the model may acknowledge that a construct could be risky, while failing to explain how attacker-controlled inputs propagate to the vulnerable operation.

We observed these forms of vague interpretation consistently across multiple datasets and models, including cases where the final prediction was correct, indicating that coincidental correctness may mask fundamentally incomplete reasoning. These observations suggest that vague interpretation stems not from an inability to describe code, but from a lack of structured guidance on which semantics matter for security. By explicitly prompting models to approximate vulnerability semantics—i.e., to identify security-relevant statements and reason about their data/control dependencies—VSP reduces the likelihood that reasoning remains purely descriptive and helps shift the model's focus toward the core mechanisms underlying exploitable behavior.

Misidentified Vulnerability Cause. Another common error is that LLMs may misidentify an incorrect vulnerability cause within the code. Figure 11 presents an example, which contains a CWE-190 (Integer Overflow) vulnerability. The issue lies at lines 78-82, specifically in calculations like `noff = ext_off + 6;`. If `ext_off` is a

Fig. 16. Incomplete CFA in vulnerability detection by GPT-3.5.

vulnerability cause within the code. Figure 11 presents an example, which contains a CWE-190 (Integer Overflow) vulnerability. The issue lies at lines 78-82, specifically in calculations like `noff = ext_off + 6;`. If `ext_off` is a

very large integer, an integer overflow can occur. However, GPT-3.5, when prompted with VSP for classification, incorrectly identifies the vulnerability as CWE-122 (Heap-based Buffer Overflow). Its reasoning points to line `r = idr_start(...)` and claims that “the code does not check whether `isoent->children.cnt` is greater than `ffmax`, which may cause a heap-based buffer overflow.”

Our analysis shows that misidentified vulnerability causes are not random mistakes, but arise from several recurring patterns in how LLMs attribute causality within code. A frequent cause of misidentification is the model’s reliance on surface-level lexical or structural cues (e.g., the presence of arithmetic operations, pointer dereferences, or loop constructs) to infer the vulnerability type. In such cases, the LLM associates these cues with commonly co-occurring CWEs, even when they are not the root cause of the vulnerability. For instance, arithmetic expressions involved in loop bounds may lead the model to predict integer overflow (e.g., CWE-190), while the actual vulnerability arises from missing bounds checks on a buffer indexed by an externally controlled value (e.g., CWE-121/122).

Another recurring pattern is incorrect localization of the vulnerability’s root cause. While the model may correctly identify a vulnerable statement (e.g., a buffer write or memory access), it often attributes the vulnerability to that statement in isolation, rather than to earlier statements that introduce unsafe values or violate required invariants. This leads to vulnerability cause misidentified when different vulnerability types share similar sinks but differ fundamentally in how unsafe states are introduced.

We also observe systematic confusion among semantically related CWEs, particularly those that share overlapping manifestations but differ in root cause. For example, the model may conflate out-of-bounds access, buffer overflow, and use-after-free vulnerabilities, as they all involve memory safety violations, despite their distinct causal mechanisms. This confusion suggests that the model recognizes the category of unsafe behavior but fails to precisely distinguish the underlying conditions that define each CWE.

Notably, such misattribution of vulnerability cause can occur even when the final CWE prediction happens to be correct, indicating that correct answers may still be supported by flawed or incomplete causal reasoning. These misidentifications are largely attributable to incomplete reconstruction of data and control dependencies that connect external inputs, intermediate computations, and vulnerable operations.

Unrelated Outputs. The most severe form of incorrect reasoning observed is unrelated outputs, where the LLM generates reasoning for a completely different, non-existent code snippet or vulnerability, entirely ignoring the provided input code. This renders the LLM’s output completely useless. Figure 12 shows the `string_crypt` function, which contains a subtle CWE-121 (Out-of-bounds write) vulnerability. However, Falcon, when given this code and VSP prompting, generates completely unrelated contents. It generates reasoning for a non-existent function `func(struct st *ptr)` and identifies a CWE-415 (Double Free) vulnerability, which has no relation to the `string_crypt` function provided in the prompt. This extreme failure to follow instructions and analyze the given code, instead generating entirely fabricated content, represents a critical limitation, particularly for smaller or less capable LLMs. Such instances underscore the need for more robust instruction-following and context-adherence mechanisms in LLMs for security-critical tasks.

7 RQ7: Challenges

In this section, we investigate the challenges in LLM-based vulnerability analysis, by examining the causes of LLMs’ failures (incorrect answers). In RQ1, we show that VSP is constantly the most effective prompting strategy, while the LLMs’ performance varied across datasets and tasks. Thus, we conduct case analysis on the incorrect predictions of VSP on all the LLMs and datasets, following a sampling process similar to that for RQ5/RQ6. We ended up inspecting 481, 457, and 387 cases for detection, classification, and repair, respectively. Based on our observations, we classify the failure causes into seven categories, as shown in Figures 17–19.

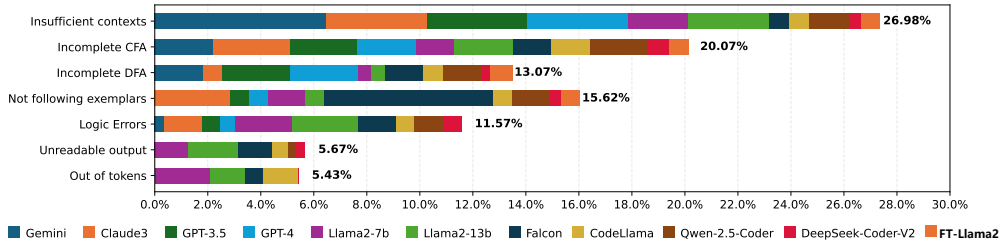


Fig. 17. Failure causes for vulnerability detection.

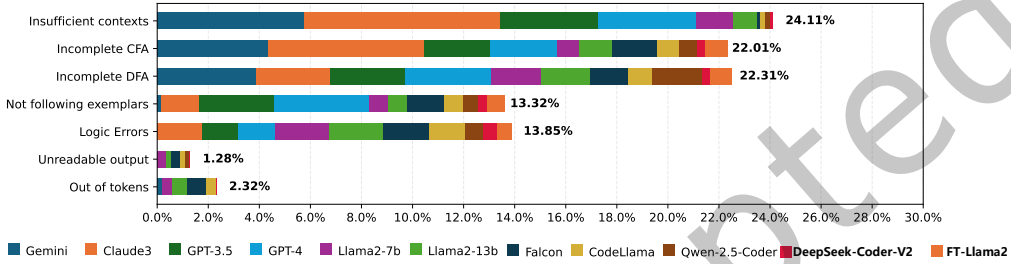


Fig. 18. Failure causes for vulnerability classification.

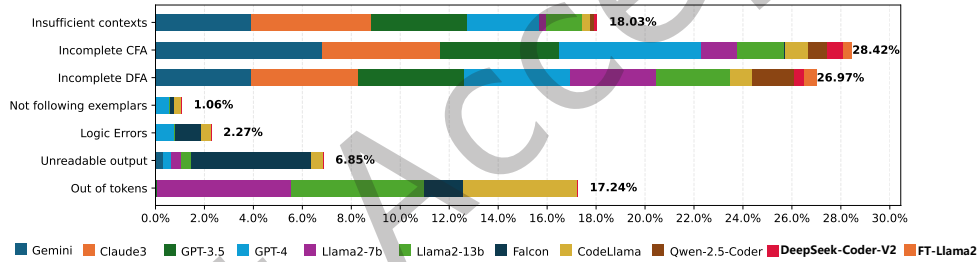


Fig. 19. Failure causes for vulnerability repair.

Insufficient Contexts. Because most LLMs have limits on #tokens processed, it is infeasible to feed them with whole projects [75]. Thus, we only input individual functions to the LLMs. However, vulnerabilities can be inter-procedural, which cannot be comprehensively analyzed without scanning contextual code in the project. Figure 23 shows an example where a function can only be used in a safe situation, but GPT-3.5 predicts it to be vulnerable because it does not know the contexts. This is common for almost all the LLMs, with a total of 18.03-26.98% proportions for the three tasks, indicating the dominance of this failure cause. For vulnerability detection, this is common for Gemini, Claude3, 2 GPTs, and 2 Llama2s, with a total of 26.98% on all the inspected failure cases, while the numbers are 24.11% and 18.03% for classification and repair, respectively.

Incomplete Control Flow Analysis (CFA). In some cases, LLMs failed due to their inability to fully capture control-flow facts necessary for correct predictions. Figure 16 shows an example with a CWE-415 double free vulnerability, which the LLM fails to detect as it misses a path along which the pointer is freed twice. While VSP has guided CFA reasoning steps and LLMs are capable of such analysis to some extent, they still fall short of very complex and comprehensive analysis. For vulnerability detection and classification, 20.07% and 22.01% of the failure cases are due to this cause, indicating it is a common challenge for LLMs. For vulnerability repair, it is even more common with a 28.42% proportion. This is because repair requires even more accurate CFA.

Across multiple incorrect and partially correct cases, we observe that incomplete CFA does not simply arise from missing individual branches, but from systematic difficulty in composing non-linear execution paths. In particular, LLMs frequently under-approximate reachability when control flow involves nested loops, early exits (e.g., break, continue, or return), or conditionals whose predicates depend on runtime values derived from external inputs. In such cases, the model may correctly reason about a single execution path while failing to recognize alternative paths under which the vulnerable statement becomes reachable.

In addition, indirect control-flow constructs—such as function pointers, callbacks, or virtual dispatch—are often treated as opaque by the model. When the concrete call target is not syntactically explicit, the LLM tends to omit the potential effects of the call from its reasoning, leading to incomplete control-flow reconstruction. These patterns explain why CFA-related errors persist even when the vulnerable statement itself is correctly identified, and why reasoning remains incomplete without explicit guidance on vulnerability-relevant control-flow paths.

Incomplete Data Flow Analysis (DFA). Similar to *incomplete CFA*, LLMs failed in some cases because they are not able to perform necessary DFA. As shown in Figure 20 as an example: when identifying a buffer overflow vulnerability, the models may ignore the vulnerable buffer write even if the index may be beyond the boundary of the buffer. This is a common failure cause with a total of 13.07-26.97% proportions for the three tasks. This indicates, again, that while LLMs are capable to some extent, they may stumble on complex and comprehensive DFA.

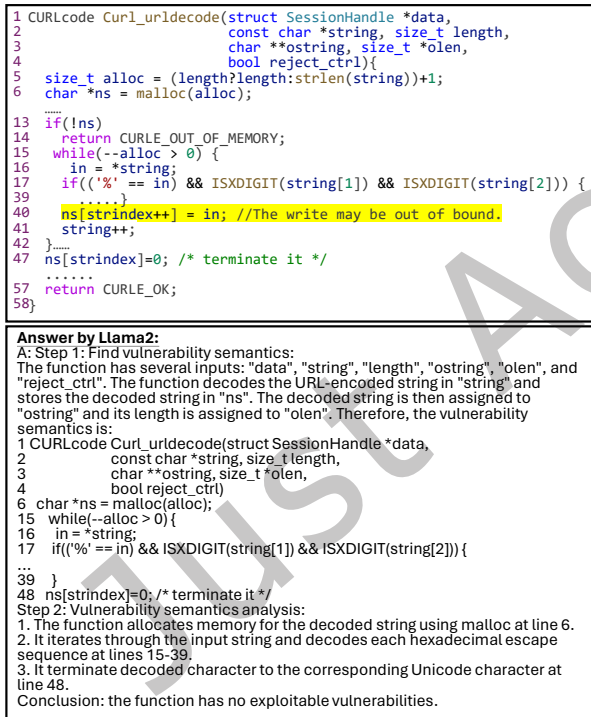


Fig. 20. Incomplete DFA in vulnerability detection by Llama2-13b.

not follow our VSP exemplars in some cases. Figure 21 shows an example: while we provide the LLMs with exemplars, the LLMs may not always follow the reasoning steps in the exemplars to analyze vulnerabilities. Instead, they act like in Standard Prompting and simply provide the answers with brief analysis. In all the

Our examination of DFA-related errors reveals that incomplete data-flow reasoning typically stems from the model's inability to track value propagation across multiple program statements and boundaries. LLMs often reason locally about individual assignments or operations, but fail to propagate constraints on externally influenced values through intermediate variables, arithmetic transformations, or conditional checks. As a result, the causal chain connecting an input source to a vulnerable sink is only partially reconstructed.

This limitation is particularly evident in the presence of pointer aliasing and shared mutable state, which are common in C/C++ code. When multiple pointers may reference the same memory location, LLMs frequently assume independence among variables, overlooking aliasing relationships that are essential for understanding how a memory access becomes unsafe. Similarly, inter-procedural data flow poses challenges: when unsafe values are introduced in one function and consumed in another, the model often fails to carry over assumptions about bounds or invariants across function calls. These factors collectively lead to misidentified vulnerability causes or incomplete vulnerability semantics, even when individual unsafe operations are correctly recognized.

Not Following Exemplars. The studied LLMs do

inspected failure cases of detection, classification, and repair, 15.62%, 13.32%, and 1.06% are in this category. Among different LLMs, this is common with Claude3, GPTs, Llama2s, and Falcon against vulnerability detection and classification. A possible reason is that these LLMs may not have sufficient instruction tuning during the training [5, 6]. In comparison, Gemini follows the exemplars in most of the cases, indicating it is good for CoT prompting.

Logic Errors. We notice that the text generated by LLMs can be logically contradictory or make incorrect logical implications during vulnerability analysis. As Figure 22 shows, an LLM may claim that an integer is checked not to be zero, but still claim that the vulnerability exists because the integer can be zero. We notice that this is common on Claude3, Llama2s, and Falcon against vulnerability detection and classification. This indicates that some of the LLMs may not be logical for effective vulnerability analysis.

Unreadable Output. Some LLMs output nothing or repeat certain text sequences in some cases. For example, the output can be like "////////////////////.....". This is mostly seen on the three least capable LLMs (Llama2s and Falcon) against the three tasks, indicating that more capable LLMs are necessary for effective vulnerability analysis.

Out of Token Limit. When an LLM is producing a result, the output may exceed its token limit, leading to output truncation, even if the reasoning steps and conclusion the model wants to generate are correct. This mostly happens in Llama2s and Falcon across the three tasks due to their smaller token limits (4,096/2,048; Table 1).

Finding 14 (RQ7): The LLMs commonly face major challenges with insufficient code contexts and incomplete control/data flow analysis in vulnerability analysis. Small models (Llama2 and Falcon) suffer from an inability to follow exemplars, making VSP much less helpful to them.

8 Discussion

8.1 Why the Performance of Existing Prompting Strategies Varies

```
// Exemplars
Q: Does the following code have exploitable vulnerabilities? Please
check whether the inputs can be maliciously used for crashing the
program.
Code to analyze ...
A: Step 1: Find vulnerability semantics: .....
Step 2: Vulnerability Semantics analysis: .....
..... (Provide a total of 20 exemplars)

// System prompt: Follow the reasoning steps and answers in the
exemplars before the last question.
// Question
Q: Does the following code have exploitable vulnerabilities? Please
check whether the inputs can be maliciously used for crashing the
program.
Code to analyze ...

// Answer by Claude3
After analyzing the code, I did not find any obvious exploitable vulnerabilities that could be
used to crash the program. The code appears to be performing various operations related to
finding the best SAS (Short Authentication String) type based on the offered and configured
algorithms. It handles different cases and checks for mandatory SAS types. The code does
not seem to have any buffer overflows, use-after-free, or other memory corruption
vulnerabilities that could lead to a crash.
```

Fig. 21. Claude3 not following exemplars in vulnerability detection.

nerability analysis.

In this case, adapting prompting strategies with explicit reasoning seems promising. However, Naive Chain-of-Thought (CoT) often fails because it focuses on general code functionality rather than the specific code behaviors that constitute vulnerability semantics. For instance, as illustrated in Figure 6, GPT-3.5 with Naive CoT merely summarizes code operations, missing the critical unchecked buffer manipulation that leads to a CWE-122 vulnerability, thus failing to provide correct reasoning or prediction.

Zero-shot completion, primarily applied to vulnerability repair, often produces patches that, while sometimes passing test cases, are functionally unreasonable or introduce new issues, indicating a lack of deep semantic

In this study, we notice that existing prompting strategies exhibit diverse effectiveness across vulnerability analysis tasks. Standard prompting relies on the LLM's inherent knowledge, often insufficient for complex vulnerability analysis. Standard few-shot learning, while guided by examples, struggles to generalize effectively due to its dependency on fixed exemplars. Retrieval-Augmented Generation (RAG) attempts to provide context through dynamic retrieval, which can be beneficial for some LLMs, but its success is contingent on the accuracy of the retrieval mechanism. Overall, these non-reasoning-based strategies frequently fall short for complex vul-

understanding of LLMs with this prompting strategy. Ultimately, the general limitations of existing strategies stem from either insufficient specific guidance for vulnerability semantics or an inability to deeply reason about complex code behaviors, leading to inconsistent and unreliable outcomes.

8.2 Why Semantics-Guided Prompting Can Help LLMs Improve

In our study, we show that our semantics-aware VSP strategy improves LLMs for vulnerability analysis of some of the evaluated LLMs. The main reason for the improvement is that it follows the human reasoning for vulnerability analysis and focuses on the parts that are most important for understanding vulnerabilities (i.e., vulnerability semantics). As Figure 6 shows, GPT-3.5 correctly detects a CWE-122 vulnerability with VSP but fails with Standard Prompting and Naive CoT—with which the LLM just simply scans the code and analyzes the major functionality of the code. Without focusing on the vulnerability semantics, it is easy to ignore the unchecked buffer manipulation at line 9. In contrast, with VSP, GPT-3.5 focuses on vulnerability semantics and notices that the write of `&s->cmdbuf[s->cmdlen]` is unchecked and may have a buffer overflow issue. Figure 8 shows another example for vulnerability repair. With VSP, GPT-3.5 successfully identifies the root cause of the buffer overflow and fixes it while standard prompt and Naive CoT simply add a boundary check without analyzing the vulnerability root cause correctly. Therefore, vulnerability-specific prompting adaptation is crucial for effective analysis.

This comparison with Naive CoT further clarifies that VSP’s effectiveness arises not merely from encouraging structured outputs, but from steering the substance of the model’s reasoning toward vulnerability-relevant code semantics. Since Naive CoT also enforces a step-by-step format but fails to achieve comparable results, we can infer that the benefit lies in the specific focus on dependency-oriented reasoning rather than the template itself. However, this reliance on vulnerability semantics focused guidance also explains why VSP does not uniformly improve performance across all models and datasets. Because the strategy depends on the model’s ability to internalize reasoning patterns from exemplars and apply them to new contexts, its success is bound by the model’s capacity to generalize beyond superficial similarities. When this generalization fails—often due to complex interprocedural flows or missing context—the model may produce structured yet substantively incomplete reasoning, consistent with our observation that VSP mitigates but does not fully eliminate reasoning errors.

8.3 Capabilities of LLMs

A: Step 1: Find vulnerability semantics:
The function has two input "x" and "y". Then, at lines 3-9, the function performs a division operation. Therefore, the vulnerability semantics is:
1 static int32_t divide(int32_t x, int32_t y){
3 if (y == 0){
5 if (x < 0)
6 return 1;
7 return -1; }
9 return x / y;
Step 2: Vulnerability semantics analysis:
At line 3-4, the function checks whether "y" is 0. If "y" is 0, the function returns 1 or -1. However, it is possible that "y" is set to 0 maliciously, which may cause a division by zero vulnerability.
Conclusion: the code has an exploitable vulnerability

Fig. 22. Logic error encountered by Llama2-13b.

Our study reveals that the inherent capabilities of LLMs, beyond just prompting strategies, influence their performance in software vulnerability analysis. This is evident in how different models, despite similar prompting, yield varied results. Larger models, such as GPT-4 and Claude3, generally demonstrate a superior capacity to analyze code security. For instance, when tasked with detecting a buffer overflow (like the CWE-122 in Figure 6) that requires tracing data flow and understanding memory operations, a larger LLM is more likely to correctly identify the intricate dependencies and potential exploit paths. In contrast, smaller models, such as Llama2-7b or Falcon, might only catch more obvious, localized issues.

The architectural choices or training methodologies of the LLMs also play a crucial role for the model effectiveness. General-purpose LLMs (e.g., GPT-4, Claude3), trained on vast and diverse text and code, often exhibit a more robust understanding of abstract reasoning and logical inference. This allows them to effectively explain

the underlying vulnerability semantics, even for complex C/C++ memory errors, by drawing on a wider range of learned patterns. In contrast, code-specific LLMs (e.g., CodeLlama, DeepSeek-Coder-V2), while trained for specific tasks like code generation, may not have sufficiently emphasized the deep semantic understanding critical for identifying subtle security vulnerabilities. Therefore, a general LLM providing a more insightful vulnerability explanation is better than a code-focused one for complex vulnerability analysis.

Finally, our results shed light on the trade-off between fine-tuning and prompt engineering for LLMs' performance on various vulnerability analysis tasks. While fine-tuning boosts the performance of open-source models (e.g., Llama2-13b) on established datasets by aligning them with specific task formats and patterns, it does not inherently solve the challenge of generalization. This suggests that the reasoning gap in fine-tuned models also need to be bridged by providing explicit vulnerability semantics focused guidance via prompting rather than relying solely on weight updates from historical data, which may not cover novel vulnerability manifestations.

8.4 Impact of Model Configurations

Beyond the external prompting strategies like VSP, our study reveals that the internal configuration of LLMs—specifically decoding strategies and system-level instructions—plays a pivotal role in the reliability of vulnerability analysis across all three tasks. Our investigation highlights the delicate balance between model creativity and the rigorous precision required for security engineering.

Regarding temperature, our overall results favor deterministic generation across detection, classification, and repair. We observed that lower temperatures (specifically 0) yield the most robust performance for all three tasks. While high temperature is often associated with creativity, in the context of vulnerability analysis, it manifests as noise. For detection and classification, higher temperatures degrade the consistency of verdicts, where a model might correctly identify a vulnerability in one run but fail in a subsequent run under identical conditions if the decoding parameters allow for variance. This stochasticity undermines the trustworthiness of LLMs as automated scanners. For repair, the negative impact of high temperature is even more pronounced because vulnerability repair requires precise syntactic adherence to the codebase. We found that increasing temperature often leads models to hallucinate non-existent library functions or generate syntactically incorrect patches, rather than finding creative solutions to the bug.

In terms of nucleus sampling (Top-P), our findings challenge the conventional wisdom that constraining the token search space always improves quality. While lowering Top-P is effective for reducing incoherence in open-ended text generation, we found that the unconstrained setting (Top-P = 1) often yields superior results, particularly for classification and repair. Vulnerability analysis relies on specific technical terminology, such as specific CWE identifiers, rare function calls, or precise patch patterns. These correct tokens may not always reside within a narrow probability mass. Artificially truncating the vocabulary distribution via a low Top-P can prevent the model from accessing the precise technical terms required for accurate classification or the specific code tokens needed for a valid patch, thereby hindering performance.

Finally, the use of system prompts to assign a persona, such as "You are a Security Expert," demonstrates a dichotomy between general-purpose and specialized models. For general-purpose models like GPT-3.5 and GPT-4, persona priming appears to activate latent domain-specific knowledge, effectively grounding the model's focus. This was particularly helpful in detection and classification on the Zero-Day dataset, where the persona helped the model navigate novel vulnerabilities without prior training data. However, this benefit does not extend to fine-tuned or code-specific models. For these models, the task definition is already internalized via their fine-tuning distribution. Introducing strong system prompts can inadvertently cause a distribution shift, interfering with the model's learned patterns and leading to performance regression, particularly in repair tasks where the model needs to strictly follow the patch format rather than engaging in conversational role-play.

8.5 Comparison Between Different Datasets

A critical observation across our experiments is the performance drop of LLMs when evaluated on the Zero-Day dataset compared to older datasets like BigVul+CVEFixes or ExtractFix. This disparity strongly suggests the presence of data leakage or contamination in the non-zero-day datasets, where LLMs may have inadvertently encountered and learned from these samples during their extensive training. The Zero-Day dataset, comprising vulnerabilities reported after the LLMs' training cutoff dates, thus serves as a more unbiased and realistic benchmark, highlighting the true generalization capabilities of these models on unseen security flaws. This performance gap underscores the imperative for future LLM-based vulnerability analysis research to prioritize evaluation against truly novel and unexposed data to ensure valid and reliable conclusions.

Conversely, when comparing performance on PrimeVul-Subset, a dataset specifically curated for its high label accuracy, against BigVul+CVEFixes, our findings present a different picture. While PrimeVul-Subset's superior ground truth quality might intuitively lead to different LLM performance, the results are mixed across models and tasks. Some LLMs perform comparably or even slightly better on BigVul+CVEFixes. This indicates that while high label accuracy is undoubtedly beneficial for evaluation, it is not the determinant of LLM effectiveness. Other factors, such as the inherent complexity, diversity, or specific characteristics of the samples within each dataset, also significantly influence the analytical capabilities. The varied outcomes suggest that LLMs' performance is impacted by multiple factors, although it is undoubted that we should use high-quality datasets for evaluation.

Furthermore, the introduction of the PrimeVul-Full dataset allows us to assess LLM generalization across a broader spectrum of vulnerability types. We observe a marked performance decline when expanding the evaluation from the five most common CWEs in PrimeVul-Subset to the 62 diverse CWEs in PrimeVul-Full. This disparity suggests that current LLMs might have been optimized for prevalent, high-frequency vulnerabilities (e.g., buffer overflows, null pointer dereferences) which are likely over-represented in their pre-training corpora. Consequently, evaluations restricted to top-tier CWEs may overestimate the practical utility of LLMs for general-purpose security auditing, emphasizing the need for benchmarks that cover the "long tail" of software security flaws.

8.6 Comparison Between Different Tasks

We notice consistent effectiveness drops from the first task to the third task. For example, the best effectiveness (i.e., F1, micro F1, and correct rate) of detection, classification, and repair against the BigVul+CVEFixes and Zero-Day datasets is 48.64/47.06% versus 34.44/25.40% versus 30.24 /20.40% with the existing prompting strategy, and 61.54/48.70% versus 47.27/40.16% versus 32.37/30.25% with VSP. The reason is that the difficulties of the three tasks increase because more detailed and accurate analysis is required. However, we also notice that on the ExtractFix dataset, the correct rates are higher. A possible reason is that the samples are relatively simple as the ExtractFix tool is a traditional technique which deals with relatively limited vulnerability repair scenarios.

8.7 Failure Causes and Recommendations

8.7.1 Insufficient Contexts. We observed that LLMs commonly fail on real-world samples due to insufficient contexts. As real-world projects are often large and involve many functions/files, vulnerabilities there are often inter-procedural. This makes vulnerability analysis difficult if we only provide a single function. However, given that LLMs can only process a limited number of tokens at a time, it is impractical to input the whole project into an LLM. **Therefore, a possible solution for this is to provide the context information (e.g., by adding code comments).** This can be achieved by again leveraging (other) LLMs to summarize the external relevant functions. Nevertheless, how to effectively prompt LLMs to generate useful external function summaries still needs to be explored.

8.7.2 Incomplete Control and Data Flow Analysis. We notice that, while LLMs have the ability to understand code semantics and perform code analysis to some extent, they fall short of conducting comprehensive control/data flow analysis, compared to conventional static/dynamic techniques. **Thus, for advanced vulnerability analysis, we recommend integrating traditional code analysis techniques with VSP-based approaches.** Specifically, static analyzers may be employed to extract the code slices dependent on external inputs. This can be helpful to improve both semantics extraction and analysis.

8.7.3 Out of Token Limit. LLMs may run out of tokens and thus be unable to output the complete analysis result. This can be due to either the input function or the result being too long. One solution here is to reduce the length of inputs and outputs. Our VSP strategy is an important step which makes the outputs more concise. However, this is not enough because we still need to input a whole function into the LLMs which may consume many tokens. **We recommend integrating program reduction techniques to reduce code length.**

<pre> 1 static void remsock(SocketInfo *f){ 2 //all the SocketInfo pointers are set to NULL after freeing. 3 if (!f) 4 return; 5 if (f->ev) 6 g_source_remove(f->ev); 7 g_free(f); } </pre>	Answer without comments: A: Step 1: Find vulnerability semantics: The function has an input "f" of type SocketInfo. At line 2, "f" is checked not to be NULL. If it is NULL, the function returns. At line 4, the member "ev" of "f" is checked not to be NULL. If it is not NULL, the function removes the event source at line 5. Finally, the pointer "f" is freed at line 6. Therefore, the vulnerability semantics is: <pre> 1 static void remsock(SocketInfo *f) 2 { 3 if (!f) 4 return; 5 g_source_remove(f->ev); 6 g_free(f); 7 } </pre> Step 2: Vulnerability semantics analysis: At line 2, "f" is checked not to be NULL. At line 4, the member "ev" of "f" is checked not to be NULL before removing the event source. At line 6, the pointer "f" is freed. However, it is not checked whether "f" has been freed before. Therefore, the code has a use-after-free (CWE-416) vulnerability. Conclusion: the code has an exploitable vulnerability.
<pre> 1 static void remsock(SocketInfo *f){ 2 //all the SocketInfo pointers are set to NULL after freeing. 3 if (!f) 4 return; 5 if (f->ev) 6 g_source_remove(f->ev); 7 g_free(f); } </pre>	Answer with comments: A: Step 1: Find vulnerability semantics: The function has an input "f" of type SocketInfo. If "f" is not NULL, the function removes the event source at line 5 and frees "f" at line 6. Therefore, the vulnerability semantics is: <pre> 1 static void remsock(SocketInfo *f) 2 { 3 if (!f) 4 return; 5 g_source_remove(f->ev); 6 g_free(f); 7 } </pre> Step 2: Vulnerability semantics analysis: At line 2, "f" is checked not to be NULL before accessing its members. At line 4, the member "ev" of "f" is checked not to be NULL before removing the event source. At line 6, "f" is freed and set to NULL, which prevents double free vulnerabilities. Conclusion: the code does not have an exploitable vulnerability.

Fig. 23. Telling LLMs the missing contexts by adding comments helps the model do correct analysis.

8.7.4 Not Following Exemplars/Logic Errors/Unreadable Output. We notice that LLMs suffer from not following exemplars, logic errors, and unreadable output issues because of their own capability limitations. Besides, we also notice the CoT-based prompting may not always work for LLMs, indicating that simple few-shot learning may not be enough. This indicates that some current LLMs may not be capable enough for effective vulnerability analysis. **Thus, using powerful LLMs is necessary for effective vulnerability analysis.**

8.8 Threats to Validity

Data Leakage/Contamination. The major threat to the validity is that the latest LLMs are trained with existing data on the Internet. Therefore, the evaluation data may have been seen by the LLMs during the training, causing data leakage/-contamination. This is a challenging problem because most powerful LLMs do not disclose training datasets, and ordinary users also do not have the capacity to train such LLMs. Existing works have noticed this issue and tried to solve this [94] by building new datasets. To mitigate this threat, we follow them and collect a dataset where samples are all published after the LLM cutoff dates.

However, even if we collected a Zero-Day dataset to conduct the experiments, the data leakage/contamination is still possible by the time we move to the next task. (e.g., the same LLM has already seen the zero-day samples during Task 1 by the time of Task 2, since we ran Task 1 before Task 2, if the testing datasets used for Task 1 and Task 2 overlap). To mitigate this possible issue, we started a completely new session for each of the sample to avoid the impact of previous analyzed samples. Also, the tasks are different. Even if the code has been seen by the LLMs, the labels/ground-truth patches are not leaked.

Reproducibility. Another threat to the validity is the result reproducibility because LLMs are statistical models and they might return different results when prompted multiple times with the same input. To mitigate this issue, we set temperature to zero to minimize the randomness of the LLMs and provide the detailed versions of the LLMs. We also mitigate this issue by using larger datasets. In this case, while the results on single samples may not be reproducible, the overall performance on the whole datasets is more reproducible.

Generalizability. We would like to note that our study currently utilizes existing datasets, which do not cover all the different types of vulnerabilities. For consistency and comparison purposes, we also had to align the CWE coverage across selections from the different original datasets. For instance, in order to compare the LLMs’ performance between our curated zero-day dataset and existing datasets, we aim to ensure they cover the same set of CWEs. Covering more or all CWEs in our datasets collected from the existing data sources implies that we need to collect the same CWEs in our zero-day dataset. However, given that our zero-day dataset can only include the most recent CVEs that are disclosed after the latest model cutoff date among the studied LLMs, it is not feasible to collect such zero-day samples of as desirably many different CWEs. Of course, we could use more CWEs in one experiment and then compare between datasets via other experiments (e.g., focusing the comparisons on the common CWEs); yet that would further complicate the presentation and discussions. Our goal was to assess how LLMs handle such complex reasoning tasks, which are often more difficult than vulnerabilities commonly found in higher-level applications (e.g., injections, XSS). While we acknowledge that these datasets do not cover the full spectrum of vulnerabilities, they offer a critical testing ground for evaluating the fundamental reasoning capabilities of LLMs. The vulnerabilities that we target are mostly in the top-25 most dangerous CWE list [1].

Besides, our work is also subject to certain limitations. Firstly, our evaluation focuses exclusively on C/C++ code and a predefined set of five Common Weakness Enumerations (CWEs) for classification and repair tasks. While C/C++ is highly vulnerable and these CWEs are among the most dangerous, our findings may not directly generalize to other programming languages or a broader spectrum of vulnerability types. Secondly, we assessed a selection of ten LLMs, prioritizing widely used and powerful versions. While we included a fine-tuned baseline for comparison, we did not explore extensive fine-tuning across all open-source models or more complex prompting strategies like Tree of Thoughts, due to practical constraints and our focus on direct prompt engineering. Lastly, while our datasets are large-scale within the chosen context (over 8,000 C/C++ samples), they do not encompass the entirety of real-world software projects, which often involve inter-procedural vulnerabilities requiring analysis beyond single functions. These deliberate choices allowed for an in-depth empirical study within manageable bounds, but they inherently define the boundaries of our conclusions.

Prompting Strategies. Although we constrain Naive CoT using systematic rules, any natural-language description of code inevitably contains some degree of abstraction. However, this limitation applies uniformly across all Naive CoT exemplars and does not affect our core conclusion: generic chain-of-thought structure alone is insufficient, whereas vulnerability-semantics-guided prompting yields consistent improvements.

Furthermore, we cannot verify whether the dependencies referenced in LLM-generated reasoning correspond to those identified by formal program analysis; accordingly, VSP should be viewed as a practical heuristic for improving reasoning relevance rather than a substitute for static or dynamic analysis.

Statistical Significance. Our empirical evaluation involved running each LLM and prompting strategy configuration once per dataset. This approach was necessitated by the extensive computational resources and time required for a study of this scale, encompassing multiple LLMs, prompting strategies, tasks, and large real-world datasets (over 8,000 samples). To minimize the inherent randomness of LLM outputs, we consistently set the temperature parameter to zero for all experiments. While this helps reduce variability, we acknowledge that without repeated experiments and formal statistical significance testing, we cannot definitively claim that every observed performance difference, particularly smaller ones, is statistically significant. Therefore, our conclusions regarding performance improvements are based on consistent trends observed across multiple LLMs, tasks, and datasets, as well as the substantial improvements in reasoning correctness, rather than statistically proven superiority in every single instance. Future work could involve repeated experiments and statistical analysis to further validate these findings.

9 Related Work

Vulnerability detection/classification. Traditionally, this problem is addressed through static [22, 45, 48] and/or dynamic code analysis [13, 91]. Respective evaluations for these techniques also exist [47, 67]. Another direction is to leverage machine learning, especially deep learning (DL) [28, 38, 60, 88, 95, 105]. Besides, there exist empirical studies evaluating and trying to improve them [15, 71].

The vulnerability detection task in this paper differs from *vulnerability identification*—asking if a given program has a specific type (CWE) of vulnerability [66], which is also a binary decision problem but with a focused search target (i.e., CWE) as an additional hint. Another related task is *vulnerability localization* [98], which aims to pinpoint the vulnerability code lines hence is intuitively more challenging for both smaller, deep-learning models [70] and larger LLMs [100]—although both can be improved via (e.g., training/fine-tuning [68] and RAG [100]) data augmentation.

Vulnerability repair. There exist traditional vulnerability repair techniques based on test cases [32] or learned patterns [10, 58]. DL-based models are also trained for vulnerability repair [17, 18, 29, 88]. Lately, Pearce et al. [75] applied several LLMs in the zero-shot setting to vulnerability repair, showing the challenges with repairing real-world vulnerabilities. Other LLM-based techniques are also proposed for vulnerability repair [36, 46, 80, 83, 96], but they require external information (e.g., error messages) and fine-tuning. In comparison, we comprehensively evaluate LLM-prompting-based approaches alongside fine-tuning. We examine various settings beyond zero-shot to determine if prompting can serve as an effective approach to improve LLM-based vulnerability repair.

In this work, we examine different prompting strategies and suggest solutions to improve them, which complement relevant other efforts such as generating training data for DL-based vulnerability repair [14, 72] and CoT exemplars for LLM-based vulnerability patching [73]—which is further enhanced through neural-symbolic reasoning synergies [74].

LLMs for vulnerability analysis. Existing studies on LLM-based vulnerability analysis have usually been conducted on small datasets, with less than 300 samples in total [84, 87] or only 30 real-world samples [87]. Ding et al. [21] built a larger dataset with 6,968 samples to evaluate LLMs on vulnerability analysis, but it only evaluates the LLMs without in-depth analysis. They also only focus on single vulnerability-analysis tasks [21, 75, 84, 87], which limits their scope. In contrast, we conduct comprehensive evaluation on LLMs for vulnerability analysis across three different tasks and prompting strategies and provide potential solutions to improve them.

10 Conclusion

We present an extensive study assessing how LLMs fare on a range of software vulnerability analysis tasks (detection, classification, and repair) versus a suite of state-of-the-art traditional approaches to these same tasks against a large set of real-world code samples. We revealed varying yet overall undesirable performance across the LLMs and identified underlying challenges, and proposed an improvement strategy via vulnerability-semantics-guided prompting (VSP). Our results indicate that VSP enhances reasoning correctness and aids prediction capabilities for some of the LLMs, signaling a potential direction for LLM-based vulnerability analysis.

Acknowledgment

We thank our associate editor and reviewers for insightful advice and constructive comments. For this work, Yu Nong, Guangbei Yi, and Haipeng Cai were supported by Army Research Office (ARO) under Grant No. W911NF21-1-002, National Science Foundation (NSF) under Grant No. CCF-2146233 and CCF-2505223, and Office of Naval Research (ONR) under Grant No. N000142212111 and 000142512252. Mohammed Aldeen and Long Cheng were supported by NSF under Grant No. 2523814, 2448205, 2239605 and 2228616. Hongxin Hu was supported by NSF under Grant No. 2228617, 2120369, and 2129164.

References

- [1] 2022. 2022 CWE Top 25 Most Dangerous Software Weaknesses. https://cwe.mitre.org/top25/archive/2022/2022_cwe_top25.html.
- [2] 2022. FlawFinder. <https://dwheeler.com/flipfinder/>.
- [3] 2024. CodeQL Documentation. <https://codeql.github.com/docs/>.
- [4] 2024. Meet Claude. <https://www.anthropic.com/claude>.
- [5] 2024. Why instruction tune LLMs? <https://www.ibm.com/topics/instruction-tuning>.
- [6] 2024. Why is Claude COMPLETELY ignoring basic instructions despite triple-mentioning them? https://www.reddit.com/r/ClaudeAI/comments/1bvoacr/why_is_claude_completely_ignoring_basic/.
- [7] Alfred V Aho, Monica S Lam, Ravi Sethi, and Jeffrey D Ullman. 2006. Compilers: Principles, Techniques, and Tools.
- [8] Masaki Aota, Hideaki Kanehara, Masaki Kubo, Noboru Murata, Bo Sun, and Takeshi Takahashi. 2020. Automation of vulnerability classification from its description using machine learning. In *2020 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 1–7.
- [9] Andrew Austin, Casper Holmgreen, and Laurie Williams. 2013. A comparison of the efficiency and effectiveness of vulnerability discovery techniques. *Information and Software Technology* 55, 7 (2013), 1279–1288.
- [10] Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to fix bugs automatically. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–27.
- [11] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: automated collection of vulnerabilities and their fixes from open-source software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*. 30–39.
- [12] Derek Bruening and Qin Zhao. 2011. Practical memory checking with Dr. Memory. In *International Symposium on Code Generation and Optimization (CGO 2011)*. IEEE, 213–223.
- [13] Juan Caballero, Gustavo Grieco, Mark Marron, and Antonio Nappa. 2012. Undangle: early detection of dangling pointers in use-after-free and double-free vulnerabilities. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis*. 133–143.
- [14] Haipeng Cai, Yu Nong, Yuzhe Ou, and Feng Chen. 2023. Generating vulnerable code via learning-based program transformations. In *AI Embedded Assurance for Cyber Systems*. Springer, 123–138.
- [15] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2021. Deep learning based vulnerability detection: Are we there yet. *IEEE Transactions on Software Engineering (TSE)* 48, 9 (2021), 3280–3296.
- [16] Yizheng Chen, Zhoujie Ding, Lamy Alowain, Xinyun Chen, and David Wagner. 2023. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. 654–668.
- [17] Zimin Chen, Steve Kommrusch, and Martin Monperrus. 2022. Neural transfer learning for repairing security vulnerabilities in c code. *IEEE Transactions on Software Engineering* 49, 1 (2022), 147–165.
- [18] Zimin Chen, Steve James Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1943–1959.
- [19] Norman Cliff. 2014. *Ordinal methods for behavioral data analysis*. Psychology Press.
- [20] Seyed Shayan Daneshvar, Yu Nong, Xu Yang, Shaowei Wang, and Haipeng Cai. 2026. VulScribeR: Exploring RAG-based vulnerability augmentation with LLMs. *ACM Transactions on Software Engineering and Methodology* 35, 5 (2026), 1–26.
- [21] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability Detection with Code Language Models: How Far Are We?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 469–481.
- [22] Navid Emamdoost, Qiushi Wu, Kangjie Lu, and Stephen McCamant. 2021. Detecting kernel memory leaks in specialized modules with ownership reasoning. In *The 2021 Annual Network and Distributed System Security Symposium (NDSS'21)*. 1–15.
- [23] Ericsson. 2023. Software vulnerability: Impact & ways to avoid it. <https://www.ericsson.com/en/security/vulnerability-management>.
- [24] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. 508–512.
- [25] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, et al. 2024. Large language models for code analysis: Do LLMs really do their job?. In *USENIX Security Symposium*. 829–846.
- [26] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30 (2020), 681–694.
- [27] Forbes Technology Council. 2023. Zero-Day Vulnerabilities: 17 Consequences And Complications. <https://www.forbes.com/sites/forbestechcouncil/2023/05/26/zero-day-vulnerabilities-17-consequences-and-complications/?sh=711e37204b41>.
- [28] Michael Fu and Chakrit Tantithamthavorn. 2022. LineVul: a transformer-based line-level vulnerability prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 608–620.

- [29] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: a T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 935–947.
- [30] Xiaoqin Fu and Haipeng Cai. 2021. FlowDist: Multi-Staged Refinement-Based Dynamic Information Flow Analysis for Distributed Software Systems. In *30th USENIX Security Symposium (USENIX Security 21)*. 2093–2110.
- [31] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, Hongyu Zhang, and Michael R Lyu. 2023. What makes good in-context demonstrations for code intelligence tasks with llms?. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 761–773.
- [32] Xiang Gao, Bo Wang, Gregory J Duck, Ruyi Ji, Yingfei Xiong, and Abhik Roychoudhury. 2021. Beyond tests: Program vulnerability repair via crash constraint extraction. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 2 (2021), 1–27.
- [33] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [34] Istvan Haller, Asia Slowinska, Matthias Neugschwandtner, and Herbert Bos. 2013. Dowsing for Overflows: A Guided Fuzzer to Find Buffer Boundary Violations. In *22nd USENIX Security Symposium (USENIX Security 13)*. USENIX Association, Washington, D.C., 49–64. <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/papers/haller>
- [35] Daniel Han and Michael Han. 2024. Unsloth. <https://github.com/unslothai/unsloth>.
- [36] Sichong Hao, Xianjun Shi, Hongwei Liu, and Yanjun Shu. 2023. Enhancing Code Language Models for Program Repair by Curricular Fine-tuning Framework. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 136–146.
- [37] Roei Hay, Omer Tripp, and Marco Pistoia. 2015. Dynamic detection of inter-application communication vulnerabilities in Android. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. 118–128.
- [38] David Hin, Andrey Kan, Huaming Chen, and M Ali Babar. 2022. LineVD: statement-level vulnerability detection using graph neural networks. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 596–607.
- [39] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [40] Ilan Peleg. 2023. The High Cost Of Security Vulnerabilities. <https://www.forbes.com/sites/forbesbusinesscouncil/2023/04/10/the-high-cost-of-security-vulnerabilities-why-observability-is-the-solution/?sh=90da08612ae6>.
- [41] Information Technology Laboratory at NIST. 2023. National Vulnerability Database (NVD) Dashboard. <https://nvd.nist.gov/general/nvd-dashboard>.
- [42] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why don't software developers use static analysis tools to find bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 672–681.
- [43] Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.
- [44] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
- [45] Daniel Kroening and Michael Tautschnig. 2014. CBMC—C bounded model checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 389–391.
- [46] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. 2024. Hybrid Automated Program Repair by Combining Large Language Models and Program Analysis. *arXiv preprint arXiv:2406.00992* (2024).
- [47] Peng Li and Baojiang Cui. 2010. A comparative study on software vulnerability static analysis techniques and tools. In *International Conference on Information Theory and Information Security*. 521–524.
- [48] Wen Li, Haipeng Cai, Yulei Sui, and David Manz. 2020. PCA: memory leak detection using partial call-path analysis. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE-Demo)*. 1621–1625.
- [49] Wen Li, Li Li, and Haipeng Cai. 2022. On the vulnerability proneness of multilingual code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 847–859.
- [50] Wen Li, Jiang Ming, Xiapu Luo, and Haipeng Cai. 2022. {PolyCruise}: A {Cross-Language} Dynamic Information Flow Analysis. In *31st USENIX Security Symposium (USENIX Security 22)*. 2513–2530.
- [51] Wen Li, Jinyang Ruan, Guangbei Yi, Long Cheng, Xiapu Luo, and Haipeng Cai. 2023. PolyFuzz: Holistic Greybox Fuzzing of Multi-Language Systems. In *32nd USENIX Security Symposium (USENIX Security 23)*.
- [52] Wen Li, Haoran Yang, Xiapu Luo, Long Cheng, and Haipeng Cai. 2023. Pyrtfuzz: Detecting bugs in python runtimes via two-level collaborative fuzzing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1645–1659.
- [53] Yi Li, Shaohua Wang, and Tien N Nguyen. 2021. Vulnerability detection with fine-grained interpretations. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 292–303.
- [54] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2244–2258.

- [55] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. In *Network and Distributed System Security (NDSS) Symposium*. 1–15.
- [56] Stephan Lipp, Sebastian Banescu, and Alexander Pretschner. 2022. An empirical study on the effectiveness of static C code analyzers for vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 544–555.
- [57] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *Comput. Surveys* 55, 9 (2023), 1–35.
- [58] Siqi Ma, Ferdian Thung, David Lo, Cong Sun, and Robert H Deng. 2017. Vurle: Automatic vulnerability detection and repair by learning from examples. In *Computer Security—ESORICS 2017: 22nd European Symposium on Research in Computer Security, Oslo, Norway, September 11–15, 2017, Proceedings, Part II* 22. Springer, 229–246.
- [59] Daniel Marjamäki. 2024. Cppcheck: a tool for static C/C++ code analysis. <https://cppcheck.sourceforge.io/>.
- [60] Yisroel Mirsky, George Macon, Michael Brown, Carter Yagemann, Matthew Pruett, Evan Downing, Sukarno Mertoguno, and Wenke Lee. 2023. VulChecker: Graph-based Vulnerability Localization in Source Code. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 6557–6574. <https://www.usenix.org/conference/usenixsecurity23/presentation/mirsky>
- [61] Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. 2024. GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models. *arXiv preprint arXiv:2410.05229* (2024).
- [62] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2450–2462 pages.
- [63] National Institute of Standards and Technology (NIST). 2022. National Vulnerability Database (NVD). <https://nvd.nist.gov>.
- [64] Nicholas Nethercote and Julian Seward. 2007. Valgrind: a framework for heavyweight dynamic binary instrumentation. *ACM Sigplan notices* 42, 6 (2007), 89–100.
- [65] Manh-Dung Nguyen, Sébastien Bardin, Richard Bonichon, Roland Groz, and Matthieu Lemerre. 2020. Binary-level Directed Fuzzing for {Use-After-Free} Vulnerabilities. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*. 47–62.
- [66] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. 2024. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. *arXiv preprint arXiv:2402.17230* (2024).
- [67] Yu Nong, Haipeng Cai, Pengfei Ye, Li Li, and Feng Chen. 2021. Evaluating and comparing memory error vulnerability detectors. *Information and Software Technology* 137 (2021), 106614.
- [68] Yu Nong, Richard Fang, Guangbei Yi, Kunsong Zhao, Xiapu Luo, Feng Chen, and Haipeng Cai. 2024. VGX: Large-Scale Sample Generation for Boosting Learning-Based Software Vulnerability Analyses. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. 1837–1849.
- [69] Yu Nong, Yuzhe Ou, Michael Pradel, Feng Chen, and Haipeng Cai. 2022. Generating realistic vulnerabilities via neural code editing: an empirical study. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1097–1109.
- [70] Yu Nong, Yuzhe Ou, Michael Pradel, Feng Chen, and Haipeng Cai. 2023. VulGen: Realistic Vulnerable Sample Generation via Pattern Mining and Deep Learning. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. 2527–2539.
- [71] Yu Nong, Rainy Sharma, Abdelwahab Hamou-Lhadj, Xiapu Luo, and Haipeng Cai. 2022. Open science in software engineering: A study on deep learning-based vulnerability detection. *IEEE Transactions on Software Engineering (TSE)* 49, 4 (2022), 1983–2005.
- [72] Yu Nong, Haoran Yang, Feng Chen, and Haipeng Cai. 2024. VinJ: An automated tool for large-scale software vulnerability data generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 567–571.
- [73] Yu Nong, Haoran Yang, Long Cheng, Hongxin Hu, and Haipeng Cai. 2024. Automated software vulnerability patching using large language models. *arXiv preprint arXiv:2408.13597* (2024).
- [74] Yu Nong, Haoran Yang, Long Cheng, Hongxin Hu, and Haipeng Cai. 2025. {APPATCH}: Automated adaptive prompting large language models for {Real-World} software vulnerability patching. In *34th USENIX Security Symposium (USENIX Security 25)*. 4481–4500.
- [75] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2339–2356.
- [76] Andre Protas and Steve Manzuik. 2023. Skeletons in Microsoft’s Closet - Silently Fixed Vulnerabilities. <https://www.blackhat.com/presentations/bh-europe-06/bh-eu-06-Manzuik.pdf>.
- [77] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [78] Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, Jeff Skowronek, and Linda Devine. 2006. Exploring methods for evaluating group differences on the NSSE and other surveys: Are the t-test and Cohen’s d indices the most appropriate choices. In *annual meeting of the Southern Association for Institutional Research*, Vol. 14. Citeseer.
- [79] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [80] Fernando Vallecillos Ruiz, Anastasiia Grishina, Max Hort, and Leon Moonen. 2024. A Novel Approach for Automatic Program Repair using Round-Trip Translation with Large Language Models. *arXiv preprint arXiv:2401.07994* (2024).

- [81] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. {AddressSanitizer}: A fast address sanity checker. In *2012 USENIX annual technical conference (USENIX ATC 12)*. 309–318.
- [82] Deepika Shanmugasundaram, Pallavi Arivukkarasu, Huaming Chen, and Haipeng Cai. 2025. Deep Learning Representations of Programs: A Systematic Literature Review. *Comput. Surveys* 58, 5 (2025), 1–37.
- [83] André Silva, Sen Fang, and Martin Monperrus. 2023. Repairllama: Efficient representations and fine-tuned adapters for program repair. *arXiv preprint arXiv:2312.15698* (2023).
- [84] Benjamin Steenhoek, Md Mahbubur Rahman, Monoshi Kumar Roy, Mirza Sanjida Alam, Earl T Barr, and Wei Le. 2024. A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection. *arXiv preprint arXiv:2403.17218* (2024).
- [85] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [86] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [87] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2024. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. In *IEEE Symposium on Security and Privacy*. 862–880.
- [88] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)* 30 (2017), 1–11.
- [89] Ronald E Walpole, Raymond H Myers, Sharon L Myers, and Keying Ye. 1993. *Probability and statistics for engineers and scientists*. Vol. 5. Macmillan New York.
- [90] Haijun Wang, Xiaofei Xie, Yi Li, Cheng Wen, Yuekang Li, Yang Liu, Shengchao Qin, Hongxu Chen, and Yulei Sui. 2020. Typestate-guided fuzzer for discovering use-after-free vulnerabilities. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 999–1010.
- [91] Ran Wang, Guangquan Xu, Xianjiao Zeng, Xiaohong Li, and Zhiyong Feng. 2018. TT-XSS: A novel taint tracking based dynamic detection framework for DOM Cross-Site Scripting. *J. Parallel and Distrib. Comput.* 118 (2018), 100–106.
- [92] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [93] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2019. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).
- [94] Yonghao Wu, Zheng Li, Jie M Zhang, and Yong Liu. 2023. Condefects: A new dataset to address the data leakage concern for llm-based fault localization and program repair. *arXiv preprint arXiv:2310.16253* (2023).
- [95] Yueming Wu, Deqing Zou, Shihan Dou, Wei Yang, Duo Xu, and Hai Jin. 2022. VulCNN: An image-inspired scalable vulnerability detection system. In *Proceedings of the 44th International Conference on Software Engineering*. 2365–2376.
- [96] Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. *arXiv preprint arXiv:2304.00385* (2023).
- [97] Guanqun Yang, Shay Dineen, Zhipeng Lin, and Xueqing Liu. 2021. Few-sample named entity recognition for security vulnerability reports by fine-tuning pre-trained language models. In *Deployable Machine Learning for Security Defense: Second International Workshop, MLHat 2021, Virtual Event, August 15, 2021, Proceedings 2*. Springer, 55–78.
- [98] Haoran Yang, Yu Nong, Tao Zhang, Xiapu Luo, and Haipeng Cai. 2024. Learning to detect and localize multilingual bugs. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2190–2213.
- [99] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [100] Guangbei Yi, Yu Nong, Minzhang Li, and Haipeng Cai. 2026. Exploring and Improving Real-World Vulnerability Data Generation via Prompting Large Language Models. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. 2044–2056.
- [101] Yuntong Zhang, Xiang Gao, Gregory J Duck, and Abhik Roychoudhury. 2022. Program vulnerability repair via inductive inference. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 691–702.
- [102] Ying Zhang, Ya Xiao, Md Mahir Asef Kabir, Danfeng Yao, and Na Meng. 2022. Example-based vulnerability detection and repair in Java code. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. 190–201.
- [103] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2024. Large Language Model for Vulnerability Detection and Repair: Literature Review and Roadmap. *arXiv preprint arXiv:2404.02525* (2024).
- [104] Xin Zhou, Ting Zhang, and David Lo. 2024. Large Language Model for Vulnerability Detection: Emerging Results and Future Directions. In *ICSE New Ideas and Emerging Results Track*. 47–51.
- [105] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in Neural Information Processing Systems (NeurIPS)* 32 (2019), 10197–10207.

- [106] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. 2024. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024).
- [107] Yoshua X ZXhang, Yann M Haxo, and Ying X Mat. 2023. Falcon LLM: A New Frontier in Natural Language Processing. *AC Investment Research Journal* 220, 44 (2023).

Just Accepted